Feistel Tools: Query-Recording and Reprogramming for QRPs

Yu-Hsuan Huang[1], Andreas Hülsing[2,4], Varun Maram[3], Silvia Ritsch[2], and Abishanka Saha[2]

[1] Max Planck Institute for Security and Privacy, Germany.
[2] Eindhoven University of Technology, The Netherlands.
[3] University of Warwick, UK.
[4] SandboxAQ, USA.
yu-hsuan.huang@mpi-sp.org, andreas@huelsing.net, msvr81@gmail.com, s.ritsch@tue.nl, a.saha1@tue.nl

Abstract. We present a new generic approach to extend proof techniques in the quantum random oracle model (QROM) — such as *query-recording* and *reprogramming* — to the quantum random permutation model (QRPM). Our framework involves simulating *bidirectional-query* random permutation oracles using *Feistel constructions* to port the QROM features for the underlying random round functions to the overall Feistel-simulated QRP. We then demonstrate the power of our framework by:

- Obtaining a tighter than state-of-the-art lower bound for the *query extrapolation problem* — a variant of which was studied by Boneh and Zhandry (Eurocrypt 2013) in the QROM — in the bidirectional QRPM, as well as recovering a meaningful lower bound for the *double-sided zero search problem* (Unruh, Asiacrypt 2023).
- Proving the adaptive zero-knowledge property of NIZKs derived from a recent variant of the Fiat-Shamir transform — called *duplex-sponge Fiat-Shamir* (Chiesa and Orrù, TCC 2025) which is deployed in the wild — in a *post-quantum* setting against *non-uniform* adversaries with preprocessing capabilities.

All in all, our work illustrates how the "*Feistel toolkit*" effectively bridges the QROM-QRPM gap by not only achieving the standard QROM techniques of query-recording and reprogramming techniques for QRPs, but also unlocking new state-of-the-art features such as proving *non-uniform security* in the QRPM.

1 Introduction

Proving security of real-world cryptographic systems in a formal manner is often a challenging task. Hence cryptographers analyze such complex systems in *idealized* models which allow to heuristically prove some meaningful security guarantees that otherwise would not have been possible in the non-idealized *standard model*. In these idealized models, we abstract the underlying public/offline primitives in the cryptographic scheme as ideal objects, or *oracles*, which can be accessed by other algorithms in our analysis — especially, the adversary. For

example, in the so-called *random oracle model (ROM)* [BR93] and *random permutation model (RPM)*, we abstract the underlying public hash functions and the underlying public permutations and ciphers as random functions and random *invertible* permutations respectively.

Focusing on the post-quantum setting where an adversary can compute offline primitives in superposition by implementing them locally on a quantum computer, the above idealized models have been generalized accordingly in the literature as the *quantum-accessible ROM (QROM)* [BDF+11] and *quantum-accessible RPM (QRPM)* wherein adversaries now have quantum access to the corresponding ideal oracles. This seemingly simple extension of the adversaries' capabilities however makes it incredibly non-trivial to extend the proof techniques from the conventional ROM and RPM to their post-quantum counterparts. To be more specific, the conventional idealized models provide the following two features — among others — which are quite handy in security proofs:

- *Query-Recording* allows us to record information about the adversaries' queries to the ideal oracles.
- *(Adaptive) Reprogramming* provides us with the flexibility to choose an oracle value $\mathcal{O}(x)$ when $\mathcal{O}$ is queried on x for the first time, without the adversary noticing this *reprogramming* of $\mathcal{O}(x)$.

Now, when the oracles can be queried in superposition, it was initially not clear how to achieve these features: for the former property, the *no-cloning principle* and the fact that measuring a quantum query can disturb an adversary's state act as *query-recording barriers*, whereas for the latter property, a superposition query can potentially *contain* all inputs thereby making it difficult to reprogram a single value $\mathcal{O}(x)$ later on without introducing inconsistencies in the adversary's view. Nonetheless, if we focus on the QROM, then several powerful techniques have been introduced in the literature to salvage this situation. Namely, Zhandry developed the so-called *compressed oracle technique* [Zha19] which could bypass the aforementioned quantum query-recording barriers, thereby allowing him and a number of follow-up works (for example, [LZ19,CFHL21,DFMS22] to name a few) to advance the state of the art in provable QROM security. Furthermore, works such as [BHH+19,GHHM21] built on top of the compressed oracle technique to achieve *tight* adaptive reprogramming in the QROM;[5] these reprogramming tools continue to be successfully employed in practice to analyze various cryptographic schemes in a post-quantum setting, including the NIST-recommended hash-based signature XMSS [CAD+20,HBG+18] and the widely-used SSH protocol [BDMX25].

However if we consider the QRPM, the situation is far from satisfactory; quantum random permutations have resisted many recent attempts to generalize the above QROM solutions to their setting. For example, prior works such as [CMSZ19,Unr21,Unr23] have tried to extend Zhandry's compressed oracle

[5] There are works which achieve (different versions of) adaptive reprogramming in the QROM without relying on compressed oracles such as [Unr14,HRS16,AHU19]; however they result in non-tight bounds.

technique to QRPs but fell short. At a high level, this has to do with the fact that, unlike (Q)ROs, the outputs of (Q)RPs are not statistically independent of each other. Nevertheless, an exciting line of new works such as [MMW25,ACMT25] have proposed fresh ideas to bring the concept of *compressed permutation oracles (CPOs)* closer to reality.CPOs enable quantum query-recording as well as adaptive reprogramming for adversaries that have oracle access to both random permutations *and their inverses*. At the same time, current CPO instantiations result in non-tight provable security bounds (see Section 1.3 for a discussion on one such CPO instantiation in a concurrent and independent work).

Importance of generic frameworks. As mentioned above, CPOs offer a generic methodology to extend the query-recording and adaptive reprogramming techniques from the QROM to the QRPM. However, there are some recent works (see Section 1.3) which achieve these techniques *separately* via tailored (non CPO-based) approaches but with *tighter* bounds than those realized by state-of-the-art CPOs. For example, [ABK$^+$24,Hos25] achieve tight adaptive reprogramming for QRPs — via their so-called *resampling lemmas* — without relying on CPOs. So one might ask: Why pursue a generic, non-tailored approach?

The main reason is that query-recording and adaptive reprogramming are not the *only* features we have currently in the QROM. There are many more advanced QROM techniques in the literature which, for example, allow us to do *straightline extraction* (i.e., given an adversary $\mathcal{A}$ which outputs a classical y that is "tightly" related to some oracle value $\mathcal{O}(x)$, we can extract x *without rewinding* the quantum $\mathcal{A}$) and analyze *non-uniform security* (i.e., against adversaries which receive *advice* from an oracle in a preprocessing phase). At the same time, one cannot expect to use the aforementioned tailored approaches to extend such powerful techniques to the QRPM because they are specific to one particular application (either query-recording or reprogramming). Hence generic frameworks (such as but not limited to CPOs) hold more promise to bridge the gap between the QROM and the QRPM in terms of available proof techniques. Hence, this motivates us to ask the following question:

> *Can we* generically *and* tightly *extend QROM proof techniques — such as* query-recording*,* adaptive reprogramming*,* and more *— to QRPM?*

1.1 Our Contributions

We propose a new generic framework inspired by *Feistel networks* to analyze quantum random permutations. Originally used in the context of blockciphers such as DES, a Feistel network (see Section 2.3) is a well-known structure to build a permutation from an arbitrary function, also called a *round function*. Our key observation is that in the QRPM, by simulating a QRP oracle using an appropriate Feistel network — namely, the three-round *permutation-Feistel-permutation* (P3FQ) construction — we can *port* the existing features of query-recording and adaptive reprogramming for the underlying round functions, when

abstracted as QROs, to the overall Feistel QRP. Moreover, by relying on the well-understood QROM features, we not only obtain *conceptually simpler* proofs — when compared to the above tailored approaches — for each of the corresponding QRPM features described below, but we also achieve *tighter* bounds than what can be realized with current CPO-based generic approaches [Car25]. Finally, the versatility of our generic Feistel framework also enables a QROM feature beyond query-recording and reprogramming — namely, proving *non-uniform security* — in the QRPM which is not known to be possible either from the aforementioned tailored approaches or existing CPO-based methods.

Query-recording via compressed Feistel oracles. We show how to bootstrap the compressed oracle technique [Zha19] to our Feistel-based QRP simulations (our "Feistel tools") to record adversaries' bidirectional queries to the QRP oracles. As an application of this P3FQ-based query-recording, we prove relevant lower bounds for certain quantum query complexity problems in the QRPM.

We first (re)prove a conjecture in [Unr21,Unr23] on invertible QRPs — i.e., the quantum hardness of the *double-sided zero search problem*. The earliest resolution of this conjecture is described in [Zha21], however it only provides a non-concrete asymptotic bound. Concrete *tight* lower bounds of $\Omega(\sqrt[4]{N})$ queries for QRPs with domain size N were later shown in [CP24,CHL+25] but via non-generic, tailored approaches. [MMW25], on the other hand, uses a generic "purified" PO (with inefficient simulation) to prove a lower bound of $\Omega(\sqrt[6]{N//(\log N)^2})$. In this work, we recover the same $\Omega(\sqrt[6]{N/(\log N)^2})$ lower bound as [MMW25] using our efficiently simulatable compressed P3FQ oracle. Towards our bounds, we extend the *transition capacity framework* [CFHL21] to our compressed Feistel oracle which enables simpler, purely classical/combinatorial reasoning.

We also obtain *new* and *tighter* query bounds — when even compared to tailored approaches — for the *query extrapolation problem*, where an attacker makes q bidirectional quantum queries to a random permutation $\mathcal{R}$ and must output $k > q$ input-output pairs of $\mathcal{R}$.[6] This problem is *intrinsically hard* to analyze because it requires finding *many* input-output pairs — a regime where previous techniques (e.g., [CHL+25,MMW25]) trivialize quickly as k grows. The only other meaningful bound for $k \geq q+1$ comes from concurrent work [Car25], which gives a loose query lower bound of order $\Omega(\sqrt[12]{N})$. In contrast, we prove a much tighter bound of order $\Omega(\sqrt{N})$, albeit only for $k \geq 4q^2+1$. Extending this tighter bound to the regime $k \leq O(q)$ appears plausible but non-trivial, and we leave this as future work.

Adaptive reprogramming of QRPs. We then show how to adaptively reprogram our P3FQ-based QRP oracle in a similar style to that for QROs in [GHHM21]. In essence, we leverage the adaptive reprogrammability of the underlying round functions when abstracted as QROs, as already *tightly* proven in [GHHM21]. By doing so, we achieve comparable tightness to the tightest known adaptive

[6] [BZ13] study a variant of this problem in the QROM where $\mathcal{R}$ is a random *function* that can only be queried in the forward direction.

reprogramming of QRPs in [ABK+24,Hos25] (except for a multiplicative factor of $\sqrt{\log N}$) obtained via tailored approaches.

As a side contribution, we use our P3FQ Feistel tool to extend a different version of adaptive reprogramming called the *measure-and-reprogram* technique [DFM20] to QRPs in Appendix G, albeit with non-tight bounds. This technique is, for example, used to prove QROM-security of the multi-round Fiat-Shamir transform. Measure-and-reprogram in the QRPM has already been achieved recently in [CHL+25] with tight bounds via a tailored approach. Nevertheless, we want to showcase the versatility of our Feistel toolkit to unlock different QROM reprogramming techniques in the QRPM in a generic manner.

Non-uniform security via reprogramming with advice. As alluded to in the previous sentence, another "stronger-than-adaptive" reprogramming technique that we unlock in the QRPM is reprogramming in the presence of adversaries with oracle-dependent *advice*. Such strong reprogramming is not known to be possible from prior adaptive reprogramming techniques in the QRPM literature (e.g., [ABK+24,Hos25]). But surprisingly, to the best of our knowledge, reprogramming with advice is also not known in the QROM literature (unlike adaptive reprogramming and measure-and-reprogram)! Hence we first establish adaptive reprogramming in the QROM with advice — also known as *auxiliary-input QROM (AI-QROM)* [HXY19] — which can be of independent interest. Then thanks to our generic Feistel framework, we are able to extend this new enhanced reprogramming from the AI-QROM to the *AI-QRPM* via the P3FQ construction. This goes on to show that our Feistel tools can not only be used to extend *existing* proof techniques in the QROM to the QRPM but also *new* QROM techniques which will potentially be introduced in the future.

As an application of our QRP adaptive reprogramming with oracle-dependent advice enabled by P3FQ, we prove *post-quantum* adaptive zero-knowledge of a recent variant of the Fiat-Shamir transformation called the *duplex-sponge Fiat-Shamir* (DSFS) [CO25] against *non-uniform* adversaries with *preprocessing* capabilities (albeit relying on sub-exponentially quantum-secure pseudorandom permutations); it is worth pointing out that such a variant has already found real-world deployment, and may even appear in a future Signal update. We also expect our improved reprogramming to be useful for other constructions such as the well-known *Even-Mansour* cipher [EM97]. Existing post-quantum security analyses of the Even-Mansour cipher (e.g., in [ABKM22,ABK+24,Hos25]) use so-called *resampling lemmas* — which can be seen a type of adaptive reprogramming of QRPs — albeit in the uniform setting. By replacing such resampling lemmas with our Feistel-based adaptive reprogramming with advice, we expect one can prove *non-uniform security* of the Even-Mansour construction — *without relying* on subexponential security of the underlying permutations.

1.2 Technical Overview

Our starting point is a recent paper [ACMT25] which proved quantum *indifferentiability* of the sponge construction. The authors introduced a Feistel-inspired

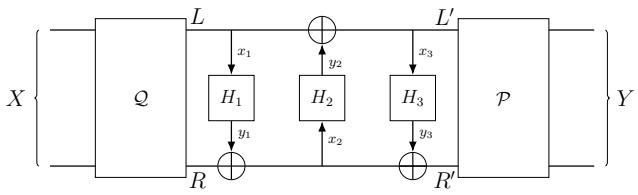

Fig. 1. Our *permutation-Feistel-permutation* P3FQ construction. Here H_1, H_2 and H_3 are round functions implemented as compressed QROs, and $\mathcal{P}$ and $\mathcal{Q}$ are masking permutations modeled as external QRPs. Also $\mathcal{Q}(X) = (L, R)$ and $\mathcal{P}(L', R') = Y$.

view of QRP oracles: any random permutation can be expressed as the "product" of another random permutation and a potentially *structured* permutation amenable to compressed oracle techniques. In [ACMT25], this structured permutation is a two-round Feistel network. Modeling the sponge's QRPs this way, they analyze its post-quantum security. However, the QRP construction in [ACMT25] is tailored to the sponge and does not yield a *general* tool for other applications.

We observe that by suitably modifying the Feistel-based QRP construction of [ACMT25], we can port QROM features — such as query-recording and reprogramming — to QRPs. In this overview, we focus on one such modification, i.e., the P3FQ construction defined in Fig. 1. P3FQ is a structured permutation realized by a 3-round Feistel cipher with QRO-modeled round functions H_1, H_2, H_3, sandwiched between two random permutations $\mathcal{P}, \mathcal{Q}$. (In contrast, [ACMT25] uses an "1FP2F" construction in our terminology.)

Given an adversary with forward and inverse access to a QRP oracle $\mathcal{O}$, we can simulate this oracle using our P3FQ construction. Since the composition of a random permutation with any (even highly structured) permutation is perfectly indistinguishable from a random permutation, we obtain the following lemma:

Lemma 1 (QRP-P3FQ indistinguishability). *For any QRPs $\mathcal{O}$, $\mathcal{P}$, and $\mathcal{Q}$, and for any adversary $\mathcal{A}$ with unbounded quantum query access, we have* $\Pr[\mathcal{A}^{\mathcal{O},\mathcal{O}^{-1}} = 1] = \Pr[\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1}} = 1]$.

Note that this only works as we do not require indifferentiability here; i.e., the adversary does not get direct access to the permutations $\mathcal{P}$ and $\mathcal{Q}$, and thereby, also not to the Feistel cipher. Otherwise, an adversary could distinguish the internal 3-round Feistel cipher from a random permutation with quantum queries using the attack in [KM10].

We will now provide a more detailed overview of the techniques used to recover the following QRPM features.

Query-Recording. We bootstrap the compressed oracle technique [Zha19] to the (bidirectional-query) QRPM via simulating the random oracles H_1, H_2, H_3 underlying P3FQ using compressed oracles, and infer the queries made to the Feistel network from queries made to the ROs. Intuitively, at any given point in time, if we identify three inputs x_1, x_2, x_3 that were queried to the oracles

H_1, H_2, H_3, and responded with y_1, y_2, y_3 respectively, then as long as the queries are "consistent", i.e. $x_1 \oplus y_2 = x_3$, we *record* $\mathcal{Q}^{-1}(x_1, y_1 \oplus x_2)$ and treat it as one of the queries that was made to $\mathsf{P3FQ}$, and responded with output $\mathcal{P}(x_3, x_2 \oplus y_3)$.

Remark 1. Intuitively, one would expect that after q queries, the total number of inputs recorded in our "compressed $\mathsf{P3FQ}$ oracle" is at most q. However, proving this appears to be quite challenging when adversaries are allowed to issue quantum queries; see Section 3.2 for further discussion. We therefore leave a rigorous proof of this "*non over-recording*" property as a meaningful open problem.

Despite the above over-recording issue, we showcase an important application of our compressed Feistel oracles: namely, bounding query complexity in the QRPM. More specifically, we obtain a new *and tighter* bound for *query extrapolation* — a natural problem that is intrinsically hard to analyze using prior techniques — in the QRPM; furthermore, by extending our notion of query-recording with the so-called *transition capacity framework*, put forward by [CFHL21] for analyzing query complexity in the QROM, we recover a meaningful query lower bound for the standard *double-sided zero search* problem [Unr23].

Adaptive Reprogramming (with Advice). We show that if an input X of the $\mathsf{P3FQ}$ construction is chosen with high min-entropy, then it is hard for an attacker to distinguish whether $\mathsf{P3FQ}(X)$ is evaluated honestly, or whether it is just a uniformly chosen value Y reprogrammed to the construction (in a suitable way). It is maybe not too surprising that our reprogramming is performed via reprogramming the underlying random oracles, and that it is bootstrapped from the reprogramming techniques considered in [GHHM21] for random functions. Furthermore, we show that such a reprogamming can be done even against *non-uniform attackers* with oracle-dependent *advice*.

Theorem (Informal version of Theorem 10)**.** *Let H_1, H_2, H_3 be random oracles $\mathcal{X} \to \mathcal{X}$, and let $\mathsf{P3FQ}^{H_1,H_2,H_3}$ be defined as in Fig. 1. Then, for any oracle algorithm with input a finite-length* advice *that arbitrarily depends on $\mathsf{P3FQ}^{H_1,H_2,H_3}$, and that makes bounded many bidirectional quantum queries to $\mathsf{P3FQ}^{H_1,H_2,H_3}$, it is hard to distinguish the following (informally specified) two oracles O_0 and O_1, which each take as input the description of a distribution $\mathcal{D}$ possessing high min-entropy. Moreover, the indistinguishability holds even if the distinguisher can program $\mathsf{P3FQ}^{H_1,H_2,H_3}(X) := Y$ for bounded many X and Y chosen adaptively.*

$O_0(\mathcal{D})$:	$O_1(\mathcal{D})$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$
2: $Y := \mathsf{P3FQ}^{H_1,H_2,H_3}(X)$	2: **reprogram** $\mathsf{P3FQ}^{H_1,H_2,H_3}(X)$ **to** $Y \leftarrow \mathcal{X}^2$
3: **return** (X, Y)	3: **return** (X, Y)

Remark 2. Our adaptive reprogramming technique is similar to the formulation in [GHHM21, Theorem 1], except that we allow adversaries to possess QRP-dependent advice, and that we do not allow $\mathcal{D}$ to produce side information about

X. This does **not** weaken our result: the bound in Theorem 10 holds information-theoretically against any query-bounded attacker, so after obtaining X from O_b, an unbounded attacker could generate any such side information themselves.

For a high-level intuition behind our proof, let us first consider the uniform setting where the adversaries do not get advice. Given an input X to the P3FQ oracle (see Fig. 1) with high min-entropy, we first argue that the underlying input x_2 to H_2 has sufficiently high min-entropy too. We show this via a careful argument using *Rényi divergence* where we rely on the independent randomness of the masking permutation $\mathcal{Q}$. This means that the RO output $H_2(x_2)$ can be adaptively reprogrammed (without advice) to a uniformly chosen value y_2 by invoking existing results in the QROM literature (i.e., [GHHM21, Theorem 1]). As $x_3 \coloneqq x_1 \oplus y_2$ is now a fresh uniformly random input to H_3, we can invoke [GHHM21, Theorem 1] once again to reprogram $H_3(x_3)$ to a uniformly chosen value y_3. All-in-all, this implies that the final reprogrammed output $\mathsf{P3FQ}(X) \coloneqq \mathcal{P}(x_3, x_2 \oplus y_3)$ is uniformly random.

Now going to the *non-uniform* setting with advice, we use the above Feistel-based approach to similarly bootstrap adaptive reprogrammability of the underlying round functions/QROs *with advice* to P3FQ. However as pointed out earlier, to the best of our knowledge, such a reprogramming of QROs with advice has not been established in the literature. Hence, we prove adaptive reprogramming in the *AI-QROM* [HXY19], which can be of independent interest, where we adapt the generic technique in [CGLQ20] that reduces security analyses in the non-uniform setting to the uniform setting via so-called *multi-instance games*.

1.3 Related Work

We will now briefly compare our Feistel toolkit with other techniques for QRPs introduced in prior work. Starting with [MMW25], the authors present a method to analyze QRPs via their so-called *strictly monotone factorization* which can be used to record queries. This, for example, allowed them to prove that the one-round *sponge construction* [BDPVA07] is unconditionally pre-image resistant in the QRPM. However, it is not clear how to use strictly monotone factorization to recover other standard QROM-based techniques such as (adaptive) reprogramming for random permutations — in contrast to our P3FQ Feistel tool — which in-turn should enable proofs of more advanced security properties (beyond pre-image resistance) for real-world permutation-based schemes.

On the other hand, the authors of [CHL+25] provide (classical-to-quantum) *lifting theorems* for establishing security in the QRPM (and the *quantum ideal cipher model*). They achieve this by first introducing a measure-and-reprogram lemma in the QRPM, and then building on top of it. In contrast, our measure-and-reprogram lemma uses a single, *unified* simulator P3FQ which also facilitates other proof techniques such as adaptive reprogramming and query-recording, which are not covered in [CHL+25].

Finally, in a concurrent and independent work, Carolan [Car25] essentially used the same P3FQ construction (called *masked Feistel* in his paper) but as

a tool to analyze a generic CPO introduced in [Unr23] that can be instantiated unconditionally. He then proved security of real-world constructions such as cryptographic sponges and *Davis-Meyer* in the QRPM, along with the hardness of double-sided zero search — albeit with non-tight bounds; i.e., the bounds in [Car25] are only meaningful when an adversary can not make more than $O(\sqrt[12]{N})$ queries to N-sized permutation oracles. In comparison, we show that one can achieve significantly tighter bounds via replacing the aforementioned CPO using our P3FQ Feistel tool. For instance, we prove a tighter bound (compared to [Car25]) for the hardness of the double-sided zero search, which stays meaningful for up-to $O(\sqrt[8]{N})$ queries. Furthermore, our P3FQ construction can also be instantiated unconditionally. To be more specific, we point out in Appendix A that the two QRP oracles ($\mathcal{P}$ and $\mathcal{Q}$) within P3FQ can be efficiently simulated unconditionally using existing techniques; this simulation however comes with a vanishingly small (statistical) additive security loss, but quantitatively it is still a much minor one when compared to the loss in [Car25].

2 Preliminaries

2.1 Basic Notations

For a positive integer n, we denote $[n]$ to be the set $\{1, \ldots, n\}$; also $(x, y) \in \{0,1\}^{2n}$ denotes the concatenation of bit strings $x, y \in \{0,1\}^n$. For a predicate P that is either true or false, we denote by its indicator $\mathbb{1}_P := 1$ if P is true, and otherwise $\mathbb{1}_P := 0$. In a probability space where P is an event, we take it as understood that $\mathbb{1}_P$ is the corresponding random variable.

For a distribution $\mathcal{D}$, we denote the sampling of a random variable X following $\mathcal{D}$ by $X \leftarrow \mathcal{D}$. Similarly, for an algorithm $\mathcal{A}$ we denote by $X \leftarrow \mathcal{A}$ sampling of X via running $\mathcal{A}$; for a finite set $\mathcal{X}$, we denote by $X \leftarrow \mathcal{X}$ sampling X *uniformly* from $\mathcal{X}$.

Let D_X and D_Y be two probability distributions over finite sets $\mathcal{X}$ and $\mathcal{Y}$ respectively. We will denote by its *tensor distribution* $\mathcal{D}_X \otimes \mathcal{D}_Y$ the distribution s.t. $\Pr_{(X,Y)\sim\mathcal{D}_X\otimes\mathcal{D}_Y}[(X,Y) = (x,y)] = \Pr_{X\sim\mathcal{D}_X}[X = x] \cdot \Pr_{Y\sim\mathcal{D}_Y}[Y = y]$, for every $(x, y) \in \mathcal{X} \times \mathcal{Y}$. In other words, it is the distribution that produces one sample from $\mathcal{D}_X$ and one from $\mathcal{D}_Y$ independently.

For two random variables X and Y, respectively following two distributions $\mathcal{D}_X$ and $\mathcal{D}_Y$ over the same finite set $\mathcal{X}$, we denote its *statistical distance* by

$$\begin{aligned}\mathsf{SD}(X,Y) &:= \frac{1}{2}\sum_{x\in\mathcal{X}}\left|\Pr_{X\sim\mathcal{D}_X}[X = x] - \Pr_{Y\sim\mathcal{D}_Y}[Y = x]\right| \\ &= \max_{f:\mathcal{X}\to\{0,1\}}\left|\Pr[f(X) = 1] - \Pr[f(Y) = 1]\right| .\end{aligned}$$

Moreover, when considering two algorithms $\mathcal{A}$ and $\mathcal{B}$ with $X \leftarrow \mathcal{A}$ and $Y \leftarrow \mathcal{B}$, we denote $\mathsf{SD}(\mathcal{A}, \mathcal{B}) := \mathsf{SD}(X, Y)$.

2.2 Guessing probability, Support, Min-entropy, Rényi divergence

Let $\mathcal{D}_X$ be a distribution over a finite set $\mathcal{X}$, and $X \sim \mathcal{D}_X$ be a random variable following $\mathcal{D}_X$. We denote its *support* and the *guessing probability* of X as

$$\underset{\mathcal{D}_X}{\mathsf{supp}}(X) := \left\{ x \in \mathcal{X} \;\middle|\; \Pr_{X \sim \mathcal{D}_X}[\, X = x \,] > 0 \right\} \text{ , and}$$

$$\underset{\mathcal{D}_X}{\mathsf{guess}}(X) := \max_{x \in \mathcal{X}} \Pr_{X \sim \mathcal{D}_X}[\, X = x \,] \text{ ,}$$

respectively, where $\mathsf{guess}_{\mathcal{D}_X}(X)$ is equal to $2^{-H_\infty(X)}$ when $H_\infty(X)$ denotes the *min-entropy* of X under $\mathcal{D}_X$. Let $\mathcal{D}_Y$ be a second distribution over $\mathcal{X}$. We denote the *Rényi divergence* of order α of $\mathcal{D}_X$ *from* $\mathcal{D}_Y$ as

$$\mathsf{Rnyi}_\alpha\big(\mathcal{D}_X \,\big\|\, \mathcal{D}_Y\big) = \left(\sum_{x \in \mathcal{X}} \frac{\Pr_{X \sim \mathcal{D}_X}[\, X = x \,]^\alpha}{\Pr_{Y \sim \mathcal{D}_Y}[\, Y = x \,]^{\alpha - 1}} \right)^{\frac{1}{\alpha - 1}} .$$

Whenever the corresponding distribution $\mathcal{D}_X$ is unambiguous, we may omit the subscript. As a short hand, when considering the conditional distribution on the event E, we denote the above by $\mathsf{supp}(X \mid E)$ and $\mathsf{guess}(X \mid E)$ respectively.

2.3 Feistel Networks

Let $\mathcal{X}$ be the set of all bit strings of a certain fixed length. For any oracle $H = (H_1, \ldots, H_n)$ with each H_i mapping from $\mathcal{X} \to \mathcal{X}$, the Feistel network is specified by

$$\mathsf{F}_n^H(L, R) := \mathsf{F}_n^{H_1, \ldots, H_n}(L, R) := \begin{cases} \big(y_L, y_R \oplus H_n(y_L)\big) & \text{if } n \text{ is odd ,} \\ \big(y_L \oplus H_n(y_R), y_R\big) & \text{if } n \text{ is even ,} \end{cases}$$

where $(y_L, y_R) := \mathsf{F}_{n-1}^{H_1, \ldots, H_{n-1}}(L, R)$ is recursively defined for every $L, R \in \mathcal{X}$, with the convention that $\mathsf{F}_0(L, R) := (L, R)$. When the context is clear, we sometimes omit the subscript n, or the superscript $H_n, \ldots, H_1$. For the most part in this paper, we consider the case of $n = 3$ unless specified otherwise, and omit the subscript via writing $\mathsf{F} := \mathsf{F}_3$. A description of F can also be found in Fig. 1 above (with the masking permutations $\mathcal{P}$ and $\mathcal{Q}$).

3 Query-Recording

We show how our *permutation-Feistel-permutation* P3FQ Feistel tool (see Fig. 1) enables an important QRPM feature: *query-recording*. In Section 3.1, we first extend Zhandry's compressed oracle technique for quantum random *functions* [Zha19] to quantum random *permutations* simulated via Feistel networks. More generally, our compressed oracle formulation applies to any "permutation construction" built from random functions, with Feistel networks as a key example.

In Section 3.2, we formally specify the *database* representation for these permutation constructions — including $\mathsf{P3FQ}$ — which records (bidirectional) quantum queries. We then illustrate a database issue called *over-recording* using $\mathsf{P2FQ}$, a 2-round variant of $\mathsf{P3FQ}$. Finally, in Sections 3.3 and 3.4, we show that despite potential over-recording, our compressed $\mathsf{P3FQ}$ oracle enables us to establish quantum query lower bounds for the *double-sided zero search problem* [Unr23] and the *query extrapolation problem* in the QRPM.

3.1 Compressed Oracle for Any Permutation Construction

We study the simulation of Feistel constructions using Zhandry's compressed oracle technique. For background on quantum mechanics and information, see [NC10]. Throughout, let $n \in \mathbb{Z}_{>0}$, let $\mathcal{X} = \{0,1\}^n$ with xor as its group operation, and let $\perp$ be a special symbol not in $\mathcal{X}$. For simplicity, we introduce our terminology abstractly. Let $\mathcal{C}^{H_1,\dots,H_n}$ be a deterministic oracle algorithm. If $\mathcal{C}$ implements a permutation — i.e., the function $f(X) := \mathcal{C}^{H_1,\dots,H_n}(X)$ is bijective with certainty over the choice of $H_1,\dots,H_n$ — then we call $\mathcal{C}$ a *permutation construction*.

To define the compressed oracle for $\mathcal{C}$, we first purify the choice of $H_1,\dots,H_n$. Namely, we consider the unitary $O[\mathcal{C}]$:

$$|X\rangle_{\mathsf{X}} \otimes |Y\rangle_{\mathsf{Y}} \otimes |H_1,\dots,H_n\rangle_{\mathsf{D}} \mapsto |X\rangle_{\mathsf{X}} \otimes \left|Y \oplus \mathcal{C}^{H_1,\dots,H_n}(X)\right\rangle_{\mathsf{Y}} \otimes |H_1,\dots,H_n\rangle_{\mathsf{D}} \ ,$$

acting on three quantum registers $\mathsf{X},\mathsf{Y},\mathsf{D}$. Intuitively, X and Y are designated query registers that the adversary can act on, and D is the oracle register (that the adversary cannot touch) consisting of n sub-registers $\mathsf{D}_1,\dots,\mathsf{D}_n$ with each D_i storing a function $\mathcal{X} \to \mathcal{X}$. For notational convenience, we will write $H = (H_1,\dots,H_n)$ as the function that coincides with H_i when restricted to $\{i\} \times \mathcal{X}$, i.e., $H(i,x) := H_i(x)$ for every $(i,x) \in [n] \times \mathcal{X}$. We will also write $\mathfrak{H}_{\mathcal{S}}^{\mathcal{Y}}$ as the set of all functions $\mathcal{S} \to \mathcal{Y}$ for any finite sets $\mathcal{S}$ and $\mathcal{Y}$, so that $H \in \mathfrak{H}_{[n]\times\mathcal{X}}^{\mathcal{X}}$ and $H_i \in \mathfrak{H}_{\mathcal{X}}^{\mathcal{X}}$ for each $i \in [n]$. Recalling the terminology used in [CFHL21], let

$$\hat{\mathcal{X}} := \{\hat{x} : \mathcal{X} \to \mathbb{C}^\times \text{ is a group homomorphism}\}^7$$

be the dual group of $\mathcal{X}$ (with the group operation being the bit-wise xor), embedded into $\mathbb{C}^{\mathcal{X}}$ via $\hat{x} \mapsto |\hat{x}\rangle := \sum_{x\in\mathcal{X}} \hat{x}(x)^* |x\rangle$. Note that the identity element $\hat{0}$ in $\hat{\mathcal{X}}$, when embedded into $\mathbb{C}^{\mathcal{X}}$, is a uniform superposition, i.e. $|\hat{0}\rangle = \sum_{x\in\mathcal{X}} |x\rangle$.

The above terminology allows us to define the compressed-oracle version of $O[\mathcal{C}]$. We define it as the unitary $\mathsf{cO}[\mathcal{C}] := \mathsf{comp}_{\mathsf{D}} \cdot O[\mathcal{C}] \cdot \mathsf{comp}_{\mathsf{D}}^\dagger$, where $\mathsf{comp}_{\mathsf{D}}$ applies Zhandry's compression isometry $\mathsf{comp} : \mathbb{C}^{\mathcal{X}} \to \mathbb{C}^{\mathcal{X}\cup\{\perp\}}$ (introduced in [Zha19] and described below) component-wise on each "sub-sub-register" $\mathsf{D}_i(x)$ of D that stores $H_i(x)$ for the state $|H_1,\dots,H_n\rangle_{\mathsf{D}}$.[8] Formally, we denote $\mathsf{comp}_{\mathsf{D}} := \bigotimes_{(i,x)\in[n]\times\mathcal{X}} \mathsf{comp}_{\mathsf{D}_i(x)}$ as

$$\mathsf{comp} := |\perp\rangle\langle\hat{0}| + \sum_{\hat{x}\in\hat{\mathcal{X}}\setminus\{\hat{0}\}} |\hat{x}\rangle\langle\hat{x}| \ .$$

[7] Here $\mathbb{C}^\times$ denotes the set of *non-zero* complex numbers.

[8] Similar to [Zha19], $\mathsf{D}_i(x) := \perp$ means that the value $H_i(x)$ has not been specified.

Indeed, it is immediate from the construction that simulating (quantum queries to) $\mathcal{C}$ using $\mathsf{cO}[\mathcal{C}]$, with the oracle register $\mathsf{D} := |\bot, \ldots, \bot\rangle_{\mathsf{D}}$ initialized to constant-$\bot$, is perfectly indistinguishable in an attacker's view.

Proposition 2. *The below are perfectly indistinguishable for any attacker $\mathcal{A}$.*

- *$\mathcal{A}$ is given unbounded many bidirectional quantum queries to $\mathcal{C}^{H_1,\ldots,H_n}$ where $H_1, \ldots, H_n$ are independent random oracles.*
- *The simulator first sets the register $\mathsf{D} := |\bot, \ldots, \bot\rangle_{\mathsf{D}}$ to be constant-$\bot$. Then $\mathcal{A}$ is allowed to make bidirectional queries to $\mathsf{cO}[\mathcal{C}]$ which takes in the XY register prepared by $\mathcal{A}$ and a (classical) bit $b \in \{0,1\}$ indicating the query direction, applies the unitary $\mathsf{cO}[\mathcal{C}^{1-2b}]$ to XYD, and returns XY to $\mathcal{A}$.*[9]

3.2 Database of Permutation Constructions

Following the above compressed oracle formulation, once an algorithm is done querying the compressed $\mathcal{C}$ oracle ($\mathsf{cO}[\mathcal{C}]$), if we measure the underlying oracle register D (in the computational basis extended by $|\bot\rangle$), then the outcome is a list of partially specified functions $D = (D_1, \ldots, D_n)$ with each $D_i = D(i, \cdot)$ — which we call a *function database* — mapping from $\mathcal{X} \to \mathcal{X} \cup \{\bot\}$, where $\bot$ indicates an unspecified value. Each function database D_i partially specifies a function $H_i : \mathcal{X} \to \mathcal{X}$ (namely, $H_i(X) = D_i(X)$ whenever $D_i(X) \neq \bot$). The tuple $D = (D_1, \ldots, D_n)$ then partially specifies the function table of the permutation construction $\mathcal{C}$, which we call a *construction database*: for any input X, $\mathcal{C}^{D_1,\ldots,D_n}(X)$ equals $\mathcal{C}^{H_1,\ldots,H_n}(X)$, except the former returns $\bot$ whenever some D_i is undefined on a value x queried during the latter evaluation (i.e., $D_i(x) = \bot$). For the rest of this section, we simply use the term "database" for either type (function or construction), when clear from context.

For notational convenience, we denote the set of all databases $\mathcal{S} \to \mathcal{Y} \cup \{\bot\}$ by $\mathfrak{D}^{\mathcal{Y}}_{\mathcal{S}}$ for any finite sets $\mathcal{S}$ and $\mathcal{Y}$, so that $D \in \mathfrak{D}^{\mathcal{X}}_{[n]\times\mathcal{X}}$ and each $D_i \in \mathfrak{D}^{\mathcal{X}}_{\mathcal{X}}$; we denote the *domain* and *image* of each database $D \in \mathfrak{D}^{\mathcal{Y}}_{\mathcal{S}}$ by $\mathsf{Dom}(D) := \{s \in \mathcal{S} \mid D(x) \neq \bot\}$ and $\mathsf{Im}(D) := \{D(x) \mid x \in \mathsf{Dom}(D)\}$ respectively. We also write $(x, y) \in D$ for $x \in \mathsf{Dom}(D)$ and $y = D(x)$, and $|D| := |\mathsf{Dom}(D)|$.

Remark 3. For a *permutation* construction $\mathcal{C}^{H_1,\ldots,H_n}$ that makes at most q_i queries to H_i, for $i \in [n]$, each time the permutation needs to be computed on an input, the corresponding compressed oracle $\mathsf{cO}[\mathcal{C}]$ can be implemented via making $2q_i$ queries to each of $\mathsf{cO}[H_i]$. The factor-2 blowup in query complexity arises from implementing the unitary $|X\rangle \otimes |H_1, \ldots, H_n\rangle \mapsto \left|\mathcal{C}^{H_1,\ldots,H_n}(X)\right\rangle \otimes |H_1, \ldots, H_n\rangle$ and uncomputing via its inverse, each using q_i queries to each of H_i. Therefore, if we measure the register D in the computational basis immediately after the q-th query to $\mathsf{cO}[\mathcal{C}]$ is responded to, the outcome $D = (D_1, \ldots, D_n)$ is such that each D_i only contains at most $2q \cdot q_i$ non-$\bot$ entries with certainty. For Feistel constructions, in particular, each $q_i = 1$.

[9] Here $\mathcal{C}^{-1}$ denotes the construction which implements the inverse of the permutation constructed by $\mathcal{C}$.

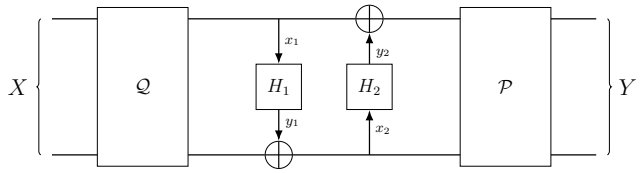

Fig. 2. The P2FQ construction with the "hard-coded" permutations $\mathcal{P}$ and $\mathcal{Q}$. Here H_1 and H_2 are (random) round functions implemented as compressed QROs.

The over-recording issue. It turns out that for certain permutation constructions, the size of the database may grow much faster than the number of (compressed-oracle) queries made by the attacker — an issue that we call *over-recording*. As an example, consider the instantiation of $\mathcal{C}$ using a 2-round variant of P3FQ, which we denote and define by $\mathsf{P2FQ}^{H_1,H_2}(X) := \mathcal{P}(\mathsf{F}_2^{H_1,H_2}(\mathcal{Q}(X)))$ for every $H_1, H_2 : \{0,1\}^n \to \{0,1\}^n$, as depicted in Fig. 2 below; here the permutations $\mathcal{P}, \mathcal{Q} : \{0,1\}^{2n} \to \{0,1\}^{2n}$ should be thought of as being "hard-coded" within $\mathcal{C}$ where they are first chosen at random.

In this case, for any input $X \in \{0,1\}^{2n}$, and for $D = (D_1, D_2) : [2] \times \{0,1\}^n \to \{0,1\}^n \cup \{\bot\}$, if there are $(x_1, y_1) \in D_1$ and $(x_2, y_2) \in D_2$ such that $\mathcal{Q}(X) = (x_1, y_1 \oplus x_2)$, then $\mathsf{P2FQ}^D(X) = \mathcal{P}((x_1 \oplus y_2), x_2)$; otherwise $\mathsf{P2FQ}^D(X) = \bot$. Therefore, the domain of the construction database $\mathsf{P2FQ}^D$ is

$$\mathsf{Dom}(\mathsf{P2FQ}^D) = \left\{ \mathcal{Q}^{-1}(x_1, y_1 \oplus x_2) \;\middle|\; \begin{array}{c} (x_1, y_1) \in \mathsf{Dom}(D_1) \\ x_2 \in \mathsf{Dom}(D_2) \end{array} \right\},$$

which is of size $|D_1| \cdot |D_2|$ as $\mathcal{Q}$ is a permutation. Now there exist algorithms making q queries to $\mathsf{cO}[\mathsf{P2FQ}^{\pm 1}]$ such that measuring the function database register D in the computational basis would produce $D = (D_1, D_2)$ with each D_i being of size up to $\Omega(q)$, which in-turn results in $\left|\mathsf{P2FQ}^D\right| \geq \Omega(q^2)$.[10]

Database of the P3FQ construction. In the case where $\mathcal{C}$ is instantiated via the P3FQ construction, the construction database $\mathsf{P3FQ}^{D_1,D_2,D_3}(X)$ for $D_1, D_2, D_3 \in \mathfrak{D}_{\mathcal{X}}^{\mathcal{X}}$ is defined to be $\mathcal{P}(x_3, x_2 \oplus y_3)$, if for some $(x_1, y_1) \in D_1$, $(x_2, y_2) \in D_2$ and $(x_3, y_3) \in D_3$ we have $x_3 = x_1 \oplus y_2$ and $\mathcal{Q}(X) = (x_1, y_1 \oplus x_2)$. Otherwise $\mathsf{P3FQ}^{D_1,D_2,D_3}(X)$ is set to be $\bot$.

Similar to the construction database of P2FQ, the size of a P3FQ database is trivially bounded by $|D_1| \cdot |D_2|$, because for every $(x_1, y_1) \in D_1$ and $(x_2, y_2) \in D_2$, there is at most one $(x_3, y_3) \in D_3$ that is "consistent" with x_1, y_1, x_2, y_2 (in the sense of $x_3 = x_1 \oplus y_2$). Since by Remark 3, after an algorithm makes q quantum bidirectional queries to $\mathsf{cO}[\mathsf{P3FQ}]$, if we measure the database registers, the outcome D_1, D_2, D_3 contains at most $2q$ entries for each D_i, we have that $\left|\mathsf{P3FQ}^{D_1,D_2,D_3}\right| \leq 4q^2$ with certainty.

[10] Such an algorithm can be easily constructed even when restricted to classical queries.

It is plausible that for an attacker querying the compressed P3FQ oracle bidirectionally, with $\mathcal{P}$ and $\mathcal{Q}$ random permutations, the over-recording issue does not arise; that is, after $q \leq \mathsf{poly}(n)$ queries and measuring $D = (D_1, D_2, D_3)$ in the database register, the size of $\mathsf{P3FQ}^D$ is at most $O(q)$ except with small probability. However, it is not clear how to prove this formally and we leave it as a meaningful open problem for future work.

3.3 Query Complexity Bound for Double-sided Zero Search

Despite the above over-recording issue, we show an important application of our compressed P3FQ oracle in-terms of providing meaningful query complexity lower bounds in the QRPM. For example, Unruh [Unr23] proposed the following conjecture about invertible quantum random permutations, which we quote below (up to a slight rephrasing):

> *Double-sided zero search:* It is hard for an adversary with bidirectional quantum query access to a random permutation $\mathcal{R}$ over $\{0,1\}^{2n}$ to find $X \in \{0,1\}^n \times \{0\}^n$ such that $\mathcal{R}(X) \in \{0,1\}^n \times \{0\}^n$.

This conjecture is a direct consequence of the reset indifferentiability of the (balanced) one-round Sponge shown in [Zha21] which, though, comes with a non-concrete, asymptotic-security bound. It has also been subsequently (re)proven by [MMW25,CP24,CHL+25]. The tightest advantage bound for an adversary making q quantum queries in finding the double-sided zero, namely $O(q^2/2^n)$, is obtained by [CP24]. However, the bound in [CP24] is obtained using a tailored simulation of the permutation. Therefore, it is meaningful for us to treat the same problem using our "general purpose" tool. In particular, we re-prove hardness of the double-sided zero search problem using our compressed P3FQ oracle wherein we obtain an advantage bound of the same order $O(q^3 n/2^n)$ as [MMW25]; our bound is also significantly tighter than the corresponding $O(q^6/2^n)$ lower bound in [Car25] which is obtained by using a different compressed permutation oracle that was introduced in [Unr23] (and whose proof of indistinguishability from a QRP required the P3FQ construction). Precisely, we prove the following result:

Theorem 3. *Let $n \in \mathbb{Z}_{>0}$, and $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$. Let $\mathcal{A}$ be an oracle algorithm given at most q bidirectional quantum queries to $\mathcal{R}$. Then for $X \leftarrow \mathcal{A}^{\mathcal{R},\mathcal{R}^{-1}}$, we have*

$$\Pr\left[\{X, \mathcal{R}(X)\} \subseteq \{0,1\}^n \times \{0\}^n\right] \leq \frac{18916(q+1)^3 n + 12}{2^n} .$$

At a high-level, our proof uses the so-called *transition capacity framework* proposed in [CFHL21]. The framework provides a powerful way to argue query complexity bounds in the QROM. First observe that P3FQ is perfectly indistinguishable from a random permutation, so swapping $\mathcal{R}$ for P3FQ does not change the statement as in Theorem 3. We then obtain the specified double-sided zero-search bound by appropriately leveraging the transition capacity framework with respect to the underlying round functions of P3FQ when abstracted as QROs. Full details of the proof can be found in Appendix B.

3.4 Query Complexity Bound for Query Extrapolation

To further demonstrate the relative strength of our Feistel tools over existing tools in the context of query recording, we consider the following query-complexity problem, a variant of which can be traced back to [BZ13].

> *The q-to-k query extrapolation problem* ($k > q$): Given q (bidirectional, quantum) queries to a random permutation $\mathcal{R}$, find $\{(X_i, Y_i)_{i\in[k]}\}$ such that $Y_i = \mathcal{R}(X_i)$ for every $i \in [k]$, and that X_i's are all distinct.

Originally, [BZ13] provided a tight bound for a variant of this problem in the QROM, where $\mathcal{R}$ is a random *function* that can only be queried in the forward direction. However, it is not so clear how to extend their technique (i.e., the so-called *rank-method*) to the QRPM where $\mathcal{R}$ is a random permutation that can be queried *bidirectionally*.

Before proceeding to our result, we first discuss limitations of prior QRPM techniques when it comes to analyzing the above q-to-k query extrapolation problem. Starting with [CP24], the work is tailored to a different problem called *double-sided zero-search* (see Section 3.3), and does not appear to be generalizable. The seemingly more extendable technique in [MMW25] also does not apply because it only considers an attacker producing a single input-output pair of $\mathcal{R}$,[11] while the quantum lifting theorem presented in [CHL+25] introduces a multiplicative factor $(8q+1)^k$ in the advantage bound that increases exponentially in k.

Using our Feistel tools, we now provide a concrete and *tighter* bound for the q-to-k query extrapolation problem, when $k \geq 4q^2 + 1$, using a relatively simple argument, thereby overcoming the above limitations.

Theorem 4. *Let $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$. Let $\mathcal{A}$ be an oracle algorithm given at most q bidirectional quantum queries to $\mathcal{R}$. Then for every $k \geq 4q^2 + 1$ and $\{(X_i, Y_i)_{i\in[k]}\} \leftarrow \mathcal{A}^{\mathcal{R},\mathcal{R}^{-1}}$, we have*

$$\Pr\left[\begin{matrix} X_i \neq X_j \quad \forall i \neq j \\ \mathcal{R}(X_i) = Y_i \; \forall i \in [k] \end{matrix}\right] \leq \frac{4q+1}{2^n} \leq O\left(\frac{q}{2^n}\right) . \tag{1}$$

Remark 4. For reference, the technique deployed concurrently in [Car25] applies whenever $k \geq q+1$, but at the same time, necessarily incurs an additive security loss of the order $\Theta(q^6/2^n)$; this bound is asymptotically less tight compared to our $O(q/2^n)$ bound for $k \geq 4q^2 + 1$. Whether or not a similar tighter bound can be achieved in the regime $k \leq O(q)$ is left as a meaningful open question.

Proof. Let $\mathcal{P}, \mathcal{Q}$ be independent random permutations over $\{0,1\}^{2n}$, and H_1, H_2, H_3 be independent random oracles $\{0,1\}^n \to \{0,1\}^n$. Switching $\mathcal{R}$ queries to P3FQ queries is perfectly indistinguishable in $\mathcal{A}$'s view. Moreover, whenever a $Y_i =$

[11] The techniques in [MMW25] may be extendable to the setting with multiple input-output pairs, but the resulting bound quickly trivializes when the number of pairs k is super-constant.

$\mathsf{P3FQ}^{H_1,H_2,H_3}(X_i)$ is found, we can extract the underlying queries to H_1 and H_2 by noticing that for $(L_i, R_i) := \mathcal{Q}(X_i)$ and $(L'_i, R'_i) = \mathsf{F}_1^{H_3} \circ \mathcal{P}^{-1}(Y_i)$, we have

$$H_1(L_i) = R_i \oplus R'_i \quad \text{and} \quad H_2(R'_i) = L_i \oplus L'_i \; .$$

Now we collect these input-output pairs of H_1 and H_2 into sets $S_1 := \{(L_i, R_i \oplus R'_i) \mid i \in [k]\}$ and $S_2 := \{(R'_i, L_i \oplus L'_i) \mid i \in [k]\}$, respectively. Since the injective mapping $\phi : S_1 \times S_2 \to \{0,1\}^{2n} \times \{0,1\}^{2n}$ defined via

$$\phi((x_1, y_1), (x_2, y_2)) := \left(\mathcal{Q}^{-1}(x_1, (y_1 \oplus x_2)), \mathcal{P} \circ \mathsf{F}_1^{H_3}((x_1 \oplus y_2), x_2)\right)$$

is such that the image $\mathsf{Im}(\phi)$ of ϕ contains $(X_i, Y_i)_{i\in[k]}$, as long as $X_1, \ldots, X_k$ are all distinct, we have $|S_1 \times S_2| = |\mathsf{Im}(\phi)| \geq \left|\{(X_i, Y_i)_{i\in[k]}\}\right| = k$. Hence, by the inequality of arithmetic and geometric means, we have

$$|S_1| + |S_2| \geq 2\sqrt{|S_1| \cdot |S_2|} = 2\sqrt{|S_1 \times S_2|} \geq 2\sqrt{k} > 4q \; ,$$

where the last inequality is via the $k > 4q^2$ assumption. This means that we can extract at least $4q + 1$ distinct input-output pairs of the random oracle H_1 and H_2 in total.

Formally, consider the following reduction algorithm $\mathcal{B}^{H_1,H_2}(\mathcal{P}, \mathcal{Q}, H_3)$ which, when given as input the entire function tables of $\mathcal{P}$, $\mathcal{Q}$, and H_3, simulates the process $\{(X_i, Y_i)_{i\in[k]}\} \leftarrow \mathcal{A}^{\mathsf{P3FQ}^{H_1,H_2,H_3}}$ via querying H_1 and H_2, and finally produces the first $4q + 1$ elements of S_1 and S_2 as specified above (if there are indeed this many). Since $\mathcal{B}$ can simulate each $\mathsf{P3FQ}$ query via 2 queries to each of H_1 and H_2, it makes no more than $4q$ queries to H_1 and H_2 in total. But now, note that $\mathcal{B}$ also succeeds in producing more than $4q$ input-output pairs of H_1 and H_2 in total whenever $\mathcal{A}$ succeeds in solving the query extrapolation problem. This is known to happen with probability at most $(4q + 1)/2^n$ (see [BZ13, Theorem 4.1]). Therefore, we have

$$\Pr\begin{bmatrix} X_i \neq X_j \quad \forall i \neq j \\ \mathcal{R}(X_i) = Y_i \; \forall i \in [k] \end{bmatrix} \leq \Pr\begin{bmatrix} |S_1| + |S_2| > 4q \\ H_1(x_1) = y_1 \; \forall (x_1, y_1) \in S_1 \\ H_2(x_2) = y_2 \; \forall (x_2, y_2) \in S_2 \end{bmatrix} \leq \frac{4q+1}{2^n} \; .$$

This concludes the proof. □

4 Adaptive Reprogramming with Advice

We will now show how our $\mathsf{P3FQ}$ construction can be adaptively reprogrammed in the presence of adversaries with oracle-dependent (classical) advice that is obtained in a preprocessing phase. As will be seen in Section 5, this technique can be used to prove post-quantum adaptive zero-knowledge of *duplex-sponge Fiat-Shamir* (DSFS) [CO25] against such *non-uniform* adversaries. It is also worth pointing out that this extension to the non-uniform setting is not known to be possible from prior tailored adaptive reprogramming techniques in the

QRPM literature (e.g., [ABK+24,Hos25]). (In Appendix C, we also prove adaptive reprogramming of P3FQ in the weaker "no advice" setting but with tighter bounds.)

Our approach is inspired by the generic technique in [CGLQ20] which reduces the task of analyzing security against non-uniform adversaries with (classical or quantum) advice in a post-quantum setting to analyzing security against *uniform* adversaries in so-called *multi-instance games*. More specifically, we first prove an adaptive reprogramming theorem for quantum *random oracles* in the non-uniform setting using ideas from [CGLQ20]; to the best of our knowledge, such a reprogramming is not known in the so-called *auxiliary-input QROM (AI-QROM)* literature (e.g., [HXY19]), and this result can be of independent interest. Then thanks to our generic Feistel framework, it is not hard to extend this adaptive reprogramming in the AI-QROM to the *AI-QRPM* via the P3FQ Feistel tool.

4.1 Adaptive Reprogramming in the AI-QROM

In the following, we consider the situation where attacker $\mathcal{A}$ is given an additional input, i.e., advice z, which is arbitrarily dependent on the random oracle.

Theorem 5. *Let $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$ be finite non-empty sets with $|\mathcal{Z}| \geq 2$, $H : \mathcal{X} \to \mathcal{Y}$ be a random oracle, z be an H-dependent random variable over $\mathcal{Z}$, and $\mathcal{A}^{H,O_b,\mathsf{Prog}}$ be an oracle algorithm given quantum query access to H, and classical query access to O_b and* Prog *(for $b \in \{0,1\}$) as specified below.*

$O_0(\mathcal{D})$:	$O_1(\mathcal{D})$:	$\mathsf{Prog}(X,Y)$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $H(X) := Y$
2: $Y := H(X)$	2: $H(X) := Y \leftarrow \mathcal{Y}$	
3: **return** (X,Y)	3: **return** (X,Y)	

Furthermore, $\mathcal{A}$ makes at most q_H queries to H in total, at most q_w queries to Prog *prior to any O_b query, and is only allowed to query O_b at most once, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X\leftarrow\mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$ for some (small but fixed) $\epsilon > 0$. Then, we have*

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0,\mathsf{Prog}}(z), \mathcal{A}^{H,O_1,\mathsf{Prog}}(z)\right) \leq 9\sqrt[3]{q_H \log|\mathcal{Z}|\,\epsilon} + 2q_w\epsilon.$$

Remark 5. Note that we forbid $\mathcal{A}(z)$ from making unbounded number of queries to the oracles H and Prog^H after the O_b^H-query, in contrast to the uniform setting as formally analyzed in Appendix C (see Lemma 14). Otherwise, say, if we set the advice z to be the xor of all of H's outputs in the truth table, then after $\mathcal{A}(z)$ queries O_b^H, it can query H on all possible inputs, take the xor of all responses, and compare the result with z. If there was no reprogramming of H (i.e., $b = 0$), then the xor must match z. Otherwise, with high probability, the xor will not match z because of H's reprogramming on a single input.

Reducing to the $q_w = 0$ case. We first show that the special case when $q_w = 0$ implies the general case. This follows the same argument as in the proof of [FFH25, Theorem 1] line-by-line.

Specifically, consider an algorithm $\mathcal{B}^{H,O_b}(z)$ that simulates $\mathcal{A}^{H,O_b,\mathsf{Prog}}(z)$, except that it book-keeps all the Prog queries locally. Note that $\mathcal{B}$ does not make any Prog query, and makes at most q_H queries to H, so $\mathsf{SD}(\mathcal{B}^{H,O_0}(z), \mathcal{B}^{H,O_1}(z))$ corresponds to the $q_w = 0$ case. Moreover, the two algorithms $\mathcal{B}^{H,O_b}(z)$ and $\mathcal{A}^{H,O_b,\mathsf{Prog}}(z)$ behave identically unless, in the O_b query, the sampled point X coincides with the Prog made by $\mathcal{A}$, which happens with probability at most $q_w\epsilon$. Hence $\mathsf{SD}(\mathcal{A}^{H,O_b,\mathsf{Prog}}, \mathcal{B}^{H,O_b}) \le q_w\epsilon$, and so

$$\begin{aligned}\mathsf{SD}\left(\mathcal{A}^{H,O_0,\mathsf{Prog}}, \mathcal{A}^{H,O_1,\mathsf{Prog}}\right) &\le \mathsf{SD}\left(\mathcal{B}^{H,O_0}, \mathcal{B}^{H,O_1}\right) + \sum_{b\in\{0,1\}} \mathsf{SD}\left(\mathcal{A}^{H,O_b,\mathsf{Prog}}, \mathcal{B}^{H,O_b}\right) \\ &\le \mathsf{SD}\left(\mathcal{B}^{H,O_0}, \mathcal{B}^{H,O_1}\right) + 2q_w\epsilon.\end{aligned}$$

We will now prove the $q_w = 0$ case, namely $\mathsf{SD}\left(\mathcal{B}^{H,O_0}, \mathcal{B}^{H,O_1}\right) \le 9\sqrt[3]{q_H \lceil \log |\mathcal{Z}| \rceil \epsilon}$. □

Proof of the $q_w = 0$ case. In the remainder, we consider the simplified case where $q_w = 0$. Since in this case $\mathcal{A}$ never queries Prog, we omit it in the superscript. Assume without loss of generality that $\mathcal{A}$ always produces a bit, and moreover it produces 1 with probability at least $\frac{1}{2}$. Then, for $(b, z^u) \leftarrow \{0,1\} \times \mathcal{Z}$ sampled uniformly and independently, we have

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{H,O_0}(z), \mathcal{A}^{H,O_1}(z)\right) = 2 \cdot \Pr\left[\, \mathcal{A}^{H,O_b}(z) = b \,\right] - 1 \\ &= 2 \cdot \mathop{\mathbb{E}}_{h\sim H}\left[\, \Pr\left[\, \mathcal{A}^{h,O_b}(z) = b \,\right] \right] - 1 \le 2 \cdot \sqrt[k]{\mathop{\mathbb{E}}_{h\sim H}\left[\Pr\left[\, \mathcal{A}^{h,O_b}(z) = b \,\right]^k \right]} - 1 \,, \\ &= 2 \cdot \sqrt[k]{\Pr\left[\, k\text{-}\mathsf{Repro}^H(\mathcal{A}, z) = 1 \,\right]} - 1 \le 2 \cdot \sqrt[k]{|\mathcal{Z}| \cdot \Pr\left[\, k\text{-}\mathsf{Repro}^H(\mathcal{A}, z^u) = 1 \,\right]} - 1\end{aligned} \tag{2}$$

where the only inequality is via Jensen's inequality, and the k-Repro game is as specified below.

k-$\mathsf{Repro}^H(\mathcal{A}, z)$:	$O_0^G(\mathcal{D})$:	$O_0^G(\mathcal{D})$:
for $i \in [k]$:	$X \leftarrow \mathcal{D}$	$X \leftarrow \mathcal{D}$
$G := H$	$Y := G(X)$	$G(X) := Y \leftarrow \mathcal{Y}$
$b_i \leftarrow \{0,1\}$	**return** (X, Y)	**return** (X, Y)
$b'_i \leftarrow \mathcal{A}^{G, O_{b_i}^G}(z)$		
return $\bigwedge_{i\in[k]} [\, b_i = b'_i \,]$		

Toward controlling the k-$\mathsf{Repro}^H(\mathcal{A}, z^u)$ game, which defines random variables $b_1, \ldots, b_k$ and $b'_1, \ldots, b'_k$, we follow the recursive approach below

$$\begin{aligned}&\Pr\left[\, k\text{-}\mathsf{Repro}^H(\mathcal{A}, z^u) \,\right] = \Pr\left[\, b_i = b'_i \;\forall i \in [k] \,\right] \\ &= \Pr\left[b_i = b'_i \;\forall i \in [k-1] \right] \cdot \Pr\left[b_k = b'_k \;\middle|\; b_i = b'_i \;\forall i \in [k-1] \right] \\ &= \Pr\left[(k-1)\text{-}\mathsf{Repro}^H(\mathcal{A}, z^u) \right] \cdot \Pr\left[b_k = b'_k \;\middle|\; b_i = b'_i \;\forall i \in [k-1] \right] \,.\end{aligned} \tag{3}$$

Moreover, we note that for every fixed choice of H, the predicate as to whether $b_i = b_i$ for each $i \in [k-1]$ can be independently simulated via making at most q_H queries to H and locally book-keeping the O_{b_i} queries. Therefore, there exists an oracle algorithm $\mathcal{B}$ that makes at most $q_{\mathcal{B}} \leq q_H \cdot (k-1)$ queries to H, such that

$$\begin{aligned}&\Pr\left[b_k = b'_k \;\middle|\; b_i = b'_i \;\forall i \in [k-1]\right] = \Pr\left[\mathcal{A}^{H,O_b}(z^u) = b \;\middle|\; \mathcal{B}^H(z^u) = 1\right] \\ &\leq \frac{1}{2} + \frac{1}{2} \cdot \mathsf{SD}\left(\mathcal{A}^{H,O_0}(z^u), \mathcal{A}^{H,O_1}(z^u) \;\middle|\; \mathcal{B}^H(z^u) = 1\right) \;,\end{aligned}$$

Deferring the final statistical distance bound to Lemma 6 below, and plugging the result back in (3), we obtain

$$\begin{aligned}\Pr\left[k\text{-}\mathsf{Repro}^H(\mathcal{A}, z^u)\right] &\leq \Pr\left[(k-1)\text{-}\mathsf{Repro}^H(\mathcal{A}, z^u)\right] \cdot \frac{1 + 2\sqrt{q_H k \cdot \epsilon}}{2} \\ &\leq \prod_{i \in [k]} \frac{1 + 2\sqrt{q_H i \cdot \epsilon}}{2} \leq \left(\frac{1 + 2\sqrt{q_H k \cdot \epsilon}}{2}\right)^k \;,\end{aligned}$$

which, when plugged back to (2) with $k := \left\lceil \sqrt[3]{(\log|\mathcal{Z}|)^2/q_H\epsilon} \right\rceil$, gives us

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{H,O_0}(z), \mathcal{A}^{H,O_1}(z)\right) \leq |\mathcal{Z}|^{1/k}\left(1 + 2\sqrt{q_H k\epsilon}\right) - 1 \\ &\leq 2^{\sqrt[3]{q_H \log|\mathcal{Z}|\epsilon}}\left(1 + 2\sqrt[3]{q_H \log|\mathcal{Z}|\,\epsilon} + 2\sqrt{q_H\epsilon}\right) - 1 \;.\end{aligned}$$

Moreover, observe that whenever $q_H \log|\mathcal{Z}|\,\epsilon \leq 1$, the above can be further simplified to

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0}(z), \mathcal{A}^{H,O_1}(z)\right) \leq 9\sqrt[3]{q_H \log|\mathcal{Z}|\,\epsilon} \;,$$

and that this anyway holds when $q_H \log|\mathcal{Z}|\,\epsilon > 1$, the proof is concluded.

□

Lemma 6. *Let $H : \mathcal{X} \to \mathcal{Y}$ be a random oracle, and $\mathcal{A}$ and $\mathcal{B}$ be oracle algorithms making at most q_H and $q_{\mathcal{B}}$ quantum queries to H respectively. Additionally, $\mathcal{A}$ is allowed to query the oracle O_b (for $b \in \{0,1\}$, as defined in Theorem 5) at most once, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X \leftarrow \mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$ for some (small but fixed) $\epsilon > 0$. Then, we have*

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0}, \mathcal{A}^{H,O_1} \;\middle|\; 1 \leftarrow \mathcal{B}^H\right) \leq 2\sqrt{(q_H + q_{\mathcal{B}})\epsilon} \;.$$

We note that the proof of [FH23, Lemma 2], which is a variant of the adaptive reprogramming [GHHM21, Theorem 1]'s proof, naturally generalizes to the setting considered here. We spell out the proof for completeness in Appendix D.

4.2 Interlude: Some Helper Lemmas on Random Permutations

Now before we proceed to extend our adaptive reprogramming of QROs with advice to QRPs via the P3FQ Feistel tool, we first establish some helper lemmas related to random permutations.

Lemma 7. *Let $X_1, \ldots, X_k \in \{0,1\}^{2n}$ be all distinct, $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$, and $\mathcal{R}_\ell(\cdot)$ be the $\mathcal{R}$-dependent function that outputs the first ℓ bits of $\mathcal{R}(\cdot)$. Then*

$$\Pr\left[\, \mathcal{R}_\ell(X_1) = \cdots = \mathcal{R}_\ell(X_k) \,\right] \leq 2^{-\ell(k-1)} \cdot \frac{1}{(1 - k \cdot 2^{-2n})^{k-1}} \,.$$

Lemma 8. *Let $\epsilon \in \mathbb{R}_{>0}$ and $X_1, \ldots, X_\alpha \in \mathcal{X}$, for a finite set $\mathcal{X}$, be mutually independent random variables such that $\mathsf{guess}(X_i) \leq \epsilon$ for each $k \in [\alpha]$. Then, for every $k \in [\alpha]$, we have*

$$\Pr\left[\, |\{X_1, \ldots, X_\alpha\}| = k \right] \leq \left\{ {\alpha \atop k} \right\} \cdot \epsilon^{\alpha - k} \,,$$

where $\left\{ {\alpha \atop k} \right\}$, called the Stirling number of second type*, denotes the number of ways the set $[\alpha]$ can be partitioned in k non-empty subsets.*

The proofs of Lemmas 7 and 8 are relatively straightforward, and can be found in Appendix E. We now turn our attention to the "main" lemma.

Lemma 9. *Let $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$, X be a random variable over $\{0,1\}^{2n}$ independent of $\mathcal{R}$, and $\mathcal{R}_n(X)$ be the last n bits of $\mathcal{R}(X)$. Then we have*

$$\mathsf{guess}(\mathcal{R}_n(X) \mid \mathcal{R}) \leq 16\sqrt{2n} \cdot \max(\mathsf{guess}(X), 2^{-n})$$

Proof. Let $\mathcal{R}_\ell(\cdot)$ be the ($\mathcal{R}$-dependent) function that, on input X, produces the last ℓ bits of $\mathcal{R}(X)$, where ℓ is chosen so that

$$2^{-\ell-1} \leq \max(\mathsf{guess}(X), 2^{-n}) \leq 2^{-\ell} \,.$$

Then, for every $\alpha \in \mathbb{Z}_{>1}$, and for $U \leftarrow \{0,1\}^\ell$ chosen uniformly and independently of $\mathcal{R}$, we immediately have that

$$\begin{aligned}
&\mathsf{guess}(\mathcal{R}_n(X) \mid \mathcal{R}) \leq \mathsf{guess}(\mathcal{R}_\ell(X) \mid \mathcal{R}) \\
&\leq \Big(\mathsf{Rnyi}_\alpha\big(\mathcal{R}_\ell(X), \mathcal{R} \,\big\|\, U, \mathcal{R}\big) \cdot \mathsf{guess}(U \mid \mathcal{R}) \Big)^{1 - \frac{1}{\alpha}} \\
&\leq \Big(2 \cdot \mathsf{Rnyi}_\alpha\big(\mathcal{R}_\ell(X), \mathcal{R} \,\big\|\, U, \mathcal{R}\big) \cdot \max(\mathsf{guess}(X), 2^{-n}) \Big)^{1 - \frac{1}{\alpha}} \,, \qquad (4)
\end{aligned}$$

where the first inequality follows from the fact that $\mathcal{R}_\ell(X)$ is a (fixed-length) suffix of $\mathcal{R}_n(X)$, the second inequality follows from the basic *probability preservation* property of Rényi divergence (see [Pre17] e.g.), and the third inequality follows from $\mathsf{guess}(U \mid \mathcal{R}) = 2^{-\ell} \leq 2 \cdot \max(\mathsf{guess}(X), 2^{-n})$.

Toward bounding the Rényi divergence in (4), we observe that

$$\begin{aligned}
&\mathsf{Rnyi}_\alpha\big(\mathcal{R}_\ell(X),\mathcal{R} \,\big\|\, U,\mathcal{R}\big)^{\alpha-1} = \sum_{u^\circ,\mathcal{R}^\circ} \frac{\Pr\left[\,\mathcal{R}_\ell(X)=u^\circ \wedge \mathcal{R}=\mathcal{R}^\circ\,\right]^\alpha}{\Pr\left[\,U=u^\circ \wedge \mathcal{R}=\mathcal{R}^\circ\,\right]^{\alpha-1}} \\
&= 2^{\ell(\alpha-1)} \sum_{u^\circ,\mathcal{R}^\circ} \Pr\left[\,\mathcal{R}=\mathcal{R}^\circ\,\right] \cdot \Pr\left[\,\mathcal{R}_\ell(X)=u^\circ \;\middle|\; \mathcal{R}=\mathcal{R}^\circ\,\right]^\alpha \\
&= 2^{\ell(\alpha-1)} \Pr\Big[\mathcal{R}_\ell(X_1)=\cdots=\mathcal{R}_\ell(X_\alpha)\Big]\,, \qquad (5)
\end{aligned}$$

where the first equality follows from the definition of Rényi divergence, and the last equality holds for $X_1,\ldots,X_\alpha$ chosen independently and identically to X.

Next, for each $k \in [\alpha]$, define the auxiliary event $E_k^\alpha : |\{X_1,\ldots,X_\alpha\}| = k$, conditioned on which we denote the first k elements of $\{X_1,\ldots,X_\alpha\}$ by $X'_1,\ldots,X'_k$ (in arbitrary order). We then bound the probability in (5) via

$$\begin{aligned}
&\Pr\Big[\mathcal{R}_\ell(X_1)=\cdots=\mathcal{R}_\ell(X_\alpha)\Big] \le \sum_{k\in[\alpha]} \Pr[E_k^\alpha] \cdot \Pr\Big[\mathcal{R}_\ell(X'_1)=\cdots=\mathcal{R}_\ell(X'_k) \;\Big|\; E_k^\alpha\Big] \\
&\le \sum_{k\in[\alpha]} \left\{{\alpha \atop k}\right\} \cdot 2^{-\ell(\alpha-k)} \cdot 2^{-\ell(k-1)} \cdot \left(\frac{1}{1-k\cdot 2^{-2n}}\right)^{k-1} \\
&\le \alpha^\alpha \cdot 2^{-\ell(\alpha-1)} \cdot \left(\frac{1}{1-\alpha\cdot 2^{-2n}}\right)^{\alpha-1}\,,
\end{aligned}$$

where the first inequality follows from the fact that the event $E_1^\alpha \vee \cdots \vee E_k^\alpha$ happens with certainty, the second inequality follows from bounding $\Pr\left[\,E_k^\alpha\,\right]$ via Lemma 8 (with $\mathsf{guess}(X_i) \le 2^{-\ell}$ for each i), and bounding the multi-collision probability $\Pr\Big[\mathcal{R}_\ell(X'_1)=\cdots=\mathcal{R}_\ell(X'_k) \;\Big|\; E_k^\alpha\Big]$ via Lemma 7, and the last inequality follows from the fact that $\sum_{k\in[\alpha]} \left\{{\alpha \atop k}\right\} \le \alpha^\alpha$. Plugging the above back to (4) and (5), we obtain

$$\mathsf{guess}(L \mid \mathcal{R}) \le \alpha \left(\frac{2\cdot\max(\mathsf{guess}(X), 2^{-n})}{1-\alpha\cdot 2^{-2n}}\right)^{1-\frac{1}{\alpha}}\,,$$

and conclude the proof by plugging in $\alpha = 2n$. □

4.3 Adaptive Reprogramming in the AI-QRPM

After establishing adaptive reprogramming for QROs in the non-uniform setting, we are now ready to prove adaptive reprogramming for our P3FQ construction against non-uniform adversaries.

Theorem 10. *Let $\mathcal{X} := \{0,1\}^n$ be a set of fixed-length strings, $\mathcal{Z}$ be a finite non-empty set with $|\mathcal{Z}| \ge 2$, $\mathcal{P}$ and $\mathcal{Q}$ be independent random permutations over $\mathcal{X}^2$, $H = (H_1, H_2, H_3)$ be so that each $H_i : \mathcal{X} \to \mathcal{X}$ is an independently sampled random function, z be any* P3FQ*-dependent random variable over $\mathcal{Z}$ (i.e. such*

that $(\mathcal{P}, \mathcal{Q}, H) \to \mathsf{P3FQ} \to z$ *forms a Markov chain), and* $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_b,\mathsf{Prog}}$ *be an oracle algorithm given (bidirectional) quantum query access to* P3FQ*, and classical query access to* O_b *and* Prog *(for* $b \in \{0,1\}$*) as specified below.*

$O_0(\mathcal{D})$:	$O_1(\mathcal{D})$:	$\mathsf{Prog}(X,Y)$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $(L,R) := \mathcal{Q}(X)$
2: $Y := \mathsf{P3FQ}^H(X)$	2: $Y \leftarrow \mathcal{X}^2$	2: $(L',R') := \mathcal{P}^{-1}(Y)$
3: **return** (X,Y)	3: $\mathsf{Prog}(X,Y)$	3: $H_2(R \oplus H_1(L)) := L' \oplus L$
	4: **return** (X,Y)	4: $H_3(L') := R' \oplus R \oplus H_1(L)$

Furthermore, $\mathcal{A}$ *makes at most* $q_{\mathcal{R}}$ *queries to* P3FQ *and* $\mathsf{P3FQ}^{-1}$ *in total, at most* q_w *queries to* Prog *prior to any* O_b *query, and is only allowed to query* O_b *at most once, with (the description of) a distribution* $\mathcal{D}$ *over* $\mathcal{X}^2$ *that satisfies* $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X\leftarrow\mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \le \epsilon$ *for some (small but fixed)* $\epsilon > 0$*; specifically,* $\mathcal{D}$ *can be described as a circuit that cannot make any queries to* P3FQ*,* $\mathsf{P3FQ}^{-1}$*, or its internal round functions. Then, we have*

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\right)\\ &\le 44\sqrt[3]{nq_{\mathcal{R}}\log|\mathcal{Z}|\,(\epsilon+2^{-n})} + 48nq_w(\epsilon+2^{-n})\,.\end{aligned}$$

We essentially bootstrap the adaptive reprogrammability of the underlying round functions/QROs $H = (H_1, H_2, H_3)$ against non-uniform attackers, that has been established in Theorem 5, as follows.

Proof of Theorem 10. Our proof proceeds through the following hybrid sequence

$$\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z) \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z) \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\,,$$

with Hyb as defined in Fig. 3, where we also rewrite O_b to a form that is more compatible with the proof. The rewriting of O_1 is done via a simple change of variables, where, instead of sampling Y uniformly and programming via $\mathsf{Prog}(X,Y)$, we uniformly sample the values y_2 and y_3 that is programmed to the last two rounds H_2 and H_3 respectively.

rewritten $O_0(\mathcal{D})$:	$\mathsf{Hyb}(\mathcal{D})$:	rewritten $O_1(\mathcal{D})$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$
2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$	2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$	2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$
3: $y_2 := H_2(R)$	3: $H_2(R) := y_2 \leftarrow \mathcal{X}$	3: $H_2(R) := y_2 \leftarrow \mathcal{X}$
4: $x_3 := L \oplus y_2$	4: $x_3 := L \oplus y_2$	4: $x_3 := L \oplus y_2$
5: $y_3 := H_3(x_3)$	5: $y_3 := H_3(x_3)$	5: $H_3(x_3) := y_3 \leftarrow \mathcal{X}$
6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$	6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$	6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$

Fig. 3. A simple hybrid sequence to reprogram P3FQ.

From O_0 to Hyb. The only difference between O_0 and Hyb is that, the former evaluates the second round $y_2 := H_2(R)$, while the latter programs the second round $H_2(R) := y_2 \leftarrow \mathcal{X}$ uniformly. Moreover, given $\mathcal{P}, \mathcal{Q}, H_1, H_3$, the $\mathsf{P3FQ}$ advice z can be simulated; also each $\mathsf{P3FQ}$ query can be simulated using 2 queries to H_2, and each Prog query can be simulated via programming H_2 once. Therefore, substituting $\mathcal{D}$ in Theorem 5 with the distribution described by $\mathcal{D}$ and $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ here, that samples L as mentioned above, we obtain

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z)\right)\\ &\leq 2q_w\,\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q}) + 9\sqrt[3]{2q_{\mathcal{R}}\lceil\log|\mathcal{Z}|\rceil\,\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q})}\,.\end{aligned}$$

Observe that $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ is independent of $\mathcal{D}$, so that conditioned on every fixed choice of $\mathcal{D}$, the distribution of the permutation $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ remains uniform. Therefore, we obtain

$$\begin{aligned}\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q}) &= \mathop{\mathbb{E}}_{\mathcal{D}^\circ \sim \mathcal{D}}\Big[\mathsf{guess}(R \mid \mathcal{D} = \mathcal{D}^\circ, \mathsf{F}_1^{H_1} \circ \mathcal{Q})\Big]\\ &\leq \mathop{\mathbb{E}}_{\mathcal{D}^\circ \sim \mathcal{D}}\Big[16\sqrt{2}n \cdot \max(\mathsf{guess}(X \mid \mathcal{D} = \mathcal{D}^\circ), 2^{-n})\Big]\\ &= 16\sqrt{2}n \cdot \big(\mathsf{guess}(X \mid \mathcal{D}) + 2^{-n}\big) \leq 16\sqrt{2}n \cdot \big(\epsilon + 2^{-n}\big)\ ,\end{aligned}$$

where the first (non-equality) inequality follows Lemma 9. Plugging it back in the above, we obtain

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z)\right)\\ &\leq 32\sqrt{2}nq_w \cdot \big(\epsilon + 2^{-n}\big) + 9\sqrt[3]{32\sqrt{2}nq_{\mathcal{R}}\lceil\log|\mathcal{Z}|\rceil \cdot (\epsilon + 2^{-n})}\,.\end{aligned}$$

From Hyb *to* O_1. The only difference between Hyb and O_1 is that, the former evaluates the third round $y_3 := H_3(x_3)$, while the latter reprograms the value $H_3(x_3) := y_3 \leftarrow \mathcal{X}$. Given that $x_3 := L \oplus y_2$ is chosen with y_2 being a freshly sampled from $\mathcal{X} = \{0,1\}^n$, Theorem 5 tells us that

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\right)\\ &\leq 2q_w \cdot 2^{-n} + 9\sqrt[3]{2q_{\mathcal{R}}\lceil\log|\mathcal{Z}|\rceil \cdot 2^{-n}}\,.\end{aligned}$$

Collecting and simplifying the bounds so far, we conclude the proof. □

Remark 6. To the best of our knowledge, there is no other existing technique that facilitates such a reprogramming lemma, be it adaptive or static, in the non-uniform setting. All prior adaptive QRP reprogramming techniques (i.e., in [ABKM22,ABK+24,Hos25]) do not work since they are tailored to the uniform setting without advice. Indeed, observe that the obtained bounds there only depend on the number of permutation queries *prior to* the reprogramming, and not on queries *afterward*. This is sufficient in the uniform setting, while in the

non-uniform setting, it is not — the attack sketched in Remark 5 would work via making unbounded number of queries *after* the reprogramming.

On the other hand, one may hope to prove such a reprogramming lemma with advice via the generic multi-instance argument (elaborated in [CGLQ20], and used in Theorem 5 for the AI-QROM). However, doing so seems to crucially rely on the existence of a compressed permutation oracle (CPO) that is (almost) *perfectly* indistinguishable. In particular, the CPO technique established in the concurrent work [Car25] comes with an additive security loss of order $\Omega(q^{12}/N)$. This would not be sufficient, as the goal in the multi-instance argument is to show that the advantage of breaking k instances simultaneously decays exponentially with k, which does not tolerate such a fixed additive loss.

Bidirectional Reprogramming on Multiple Inputs. As a bonus, in Appendix F, we show how to extend our Feistel reprogramming theorem with advice to a setting where the $\mathsf{P3FQ}$ construction can be reprogrammed *adaptively* in both forward and backward directions on *multiple* inputs.

5 Non-uniform Security of Duplex-Sponge Fiat-Shamir

An example application of our $\mathsf{P3FQ}$ adaptive reprogramming theorem with advice is proving *post-quantum* adaptive zero-knowledge of NIZKs obtained via a variant of the Fiat-Shamir transformation (FS), called duplex-sponge Fiat-Shamir (DSFS) as recently analyzed in [CO25], against *non-uniform* adversaries. As explained in [CO25], this construction is already used in practice and may appear in a future Signal update. Despite recent progress on the quantum security of sponge constructions [CP24,MMW24,ACMT25], the quantum security of DSFS — especially against *preprocessing attacks* — remains an open problem. Analogously to standard FS, DSFS turns any public-coin proof system $\Sigma = (P_\Sigma, V_\Sigma)$ into a non-interactive proof system $\mathsf{DSFS}[\Sigma, \mathcal{R}]$ using the duplex-sponge construction [BDPV12].

The sponge construction makes use of a permutation $\mathcal{R}$. For positive integer parameters $r < 2n$, states $s_i \in \{0,1\}^{2n}$, and inputs $a_i \in \{0,1\}^r$ the sponge defines two operations:

$$\begin{aligned} s_{i+1} &:= \mathsf{Sponge}^{\mathcal{R}}.\mathsf{absorb}(a_i; s_i) = \mathcal{R}((a_i \| 0^{2n-r}) \oplus s_i) \text{ , and} \\ \mathsf{ch}_i &:= \mathsf{Sponge}^{\mathcal{R}}.\mathsf{squeeze}(s_i) = s_i|_{1,\ldots,r} \text{ ,} \end{aligned} \tag{6}$$

where $x|_{1,\ldots,r}$ refers to the first r bits of x. The initial state is $s_1 = 0^{2n}$. In duplex mode, it is allowed to interleave absorb and squeeze, but it is only allowed to squeeze a state that was produced by absorb (and thereby only allowed to squeeze a state once). As a shorthand for the combined action of first absorbing a_i into s_i and then squeezing the resulting state s_{i+1} to get ch_i, we define $(\mathsf{ch}_i; s_{i+1}) := \mathsf{Sponge}^{\mathcal{R}}(a_i; s_i)$. For more details on the duplex mode, we refer to [BDPV12].

The key difference between DSFS and FS is that, instead of making independent hash calls, DSFS maintains a sponge state s, absorbing each prover

$\Sigma(\mathrm{x}, \mathrm{w})$:
1: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
2: **for** $i = 1, \ldots, k-1$
3: $\quad \mathsf{ch}_i \leftarrow V_\Sigma(a_i)$
4: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
5: **return** $(a_1, \ldots, a_k, \mathsf{ch}_1 \ldots, \mathsf{ch}_{k-1})$

$\mathsf{Ver}^{\mathcal{R}}(\mathrm{x}, \tau, a_1, \ldots, a_k)$:
1: $s_0 := \mathsf{Sponge}^{\mathcal{R}}.\mathsf{absorb}(\mathrm{x}; 0)$
2: $s_1 := \mathsf{Sponge}^{\mathcal{R}}.\mathsf{absorb}(\tau; s_0)$
3: **for** $i = 1, \ldots, k-1$
4: $\quad (\mathsf{ch}_i; s_{i+1}) := \mathsf{Sponge}^{\mathcal{R}}(a_i; s_i)$
5: **return** $V_\Sigma(\mathrm{x}, a_1, \ldots, a_k, \mathsf{ch}_1 \ldots, \mathsf{ch}_{k-1})$

$\mathsf{Prv}^{\mathcal{R}}(\mathrm{x}, \mathrm{w})$:
1: $\tau \leftarrow \{0,1\}^r$
2: $s_0 := \mathsf{Sponge}^{\mathcal{R}}.\mathsf{absorb}(\mathrm{x}; 0)$
3: $s_1 := \mathsf{Sponge}^{\mathcal{R}}.\mathsf{absorb}(\tau; s_0)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad (\mathsf{ch}_i; s_{i+1}) := \mathsf{Sponge}^{\mathcal{R}}(a_i; s_i)$
7: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
8: **return** $(\tau, a_1, \ldots, a_k)$

Fig. 4. The Duplex-Sponge Fiat-Shamir Transform (DSFS), transforming any k-round public-coin proof system $\Sigma = (P_\Sigma, V_\Sigma)$ into a non-interactive one $\mathsf{DSFS}[\Sigma, \mathcal{R}] = (\mathsf{Prv}, \mathsf{Ver})$, where $\mathcal{R}$ is a cryptographic permutation. Here $\mathsf{st_i}$'s are prover $\mathcal{P}_\Sigma$'s states.

message a_i into s and squeezing out challenges ch as needed. To make challenges for different statements x independent and s unpredictable, the verifier first absorbs the statement x and a fresh salt $\tau \leftarrow \{0,1\}^r$ into s. The proof is then $\pi = (\tau, a_1, \ldots, a_k)$. As in FS, the verifier on input (x, π) recomputes all challenges and runs $b \leftarrow V_\Sigma(x, a_1, \ldots, a_k, \mathsf{ch}_1, \ldots, \mathsf{ch}_k)$ to decide acceptance. For simplicity, we assume all absorbed messages and squeezed outputs are r-bit strings. We describe DSFS in more detail in Fig. 4. We will require that Σ satisfies the standard statistical honest-verifier zero-knowledge (HVZK) property with some error ζ_Σ, meaning there exists an efficient algorithm Sim_Σ such that on input any fixed (valid) instance-witness pair (x, w), we have

$$\mathsf{SD}\left(\Sigma(\mathrm{x}, \mathrm{w}), \mathsf{Sim}_\Sigma(\mathrm{x})\right) \leq \zeta_\Sigma \ . \tag{7}$$

5.1 Adaptive Zero-Knowledge Against Non-uniform Attackers

We now show that if $\mathcal{R}$ is modeled as a QRP, the non-interactive proof system $\mathsf{DSFS}[\Sigma, \mathcal{R}]$ satisfies the adaptive zero-knowledge property against quantum attackers with $\mathcal{R}$-dependent advice that are given bidirectional quantum oracle access to the permutation $\mathcal{R}$, with an error that is closely related to ζ_Σ, the HVZK-error of Σ. For the definition of adaptive zero-knowledge, we consider the natural generalization of [CO25] in the non-uniform setting.

Specifically, for any adversary $\mathcal{A}(z)$ given an oracle-dependent advice z, our goal is to show the existence of another pair of advices $(z_\mathcal{A}, z_S)$, and an efficient stateful simulator Sim that is in charge of simulating Prv and $\mathcal{R}$, such that $\mathcal{A}$ can not distinguish whether it is given z and is interacting with $\mathsf{Prv}^{\mathcal{R}}$ and $\mathcal{R}$, or whether it is given $z_\mathcal{A}$ and is interacting with the simulator Sim that is given z_S. Denoting the interfaces that Sim provides for Prv and $\mathcal{R}$ by $\mathsf{Sim}^{z_S}_{\mathsf{Prv}}$ and $\mathsf{Sim}^{z_S}_{\mathcal{R}}$ respectively, with the superscript indicating the advice given to Sim, we

define the advantage of adaptive zero-knowledge in this non-uniform setting as $\mathbf{Adv}^{\mathsf{NIZK},\mathsf{Sim}}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{A}, z, z_{\mathcal{A}}, z_S)$

$$:= \mathsf{SD}\left(\left(b \;\middle|\; \begin{array}{l}(\mathrm{x},\mathrm{w},aux) \leftarrow \mathcal{A}^{\mathcal{R}}(z)\\ \pi \leftarrow \mathsf{Prv}^{\mathcal{R}}(\mathrm{x},\mathrm{w})\\ b \leftarrow \mathcal{A}^{\mathcal{R}}(aux,\pi)\end{array}\right), \left(b \;\middle|\; \begin{array}{l}(\mathrm{x},\mathrm{w},aux) \leftarrow \mathcal{A}^{\mathsf{Sim}^{z_S}_{\mathcal{R}}}(z_{\mathcal{A}})\\ \pi \leftarrow \mathsf{Sim}^{z_S}_{\mathsf{Prv}}(\mathrm{x})\\ b \leftarrow \mathcal{A}^{\mathsf{Sim}^{z_S}_{\mathcal{R}}}(aux,\pi)\end{array}\right)\right) . \quad (8)$$

More compactly, we denote the above experiments in $\mathsf{SD}(\cdot,\cdot)$ by $\mathcal{A}^{\mathcal{R},\mathsf{Prv}}(z)$ and $\mathcal{A}^{\mathsf{Sim}^{z_S}_{\mathcal{R}},\mathsf{Sim}^{z_S}_{\mathsf{Prv}}}(z_{\mathcal{A}})$, with a slight abuse of notation. We say that $\mathsf{DSFS}[\Sigma,\mathcal{R}]$ achieves non-uniform adaptive zero-knowledge if for every attacker $\mathcal{A}$ given the advice z, there is indeed such a pair of simulated advices $(z_{\mathcal{A}}, z_S)$, and a simulator Sim for which $\mathbf{Adv}^{\mathsf{NIZK},\mathsf{Sim}}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{A}, z, z_{\mathcal{A}}, z_S)$ is negligibly small, and the total running time of $\mathcal{A}^{\mathsf{Sim}^{z_S}_{\mathcal{R}},\mathsf{Sim}^{z_S}_{\mathsf{Prv}}}(z_{\mathcal{A}})$ is similar to the total running time of $\mathcal{A}$ when interacting with Prv and $\mathcal{R}$. We prove the following result:

Theorem 11 (Non-uniform adaptive zero-knowledge of DSFS). *Let $\mathcal{Z}$ be a finite set with $|\mathcal{Z}| \geq 2$, r and n be positive integers such that $r < 2n$, and $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$. Let $k \in \mathbb{N}$, and Σ be a $(2k-1)$-round public-coin proof-system with HVZK-error ζ_Σ. Assume the existence of a sub-exponentially secure quantum PRP (qPRP), the advantage of which we denote by $\mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{B})$ for any attacker $\mathcal{B}$.*

Then, for every $\delta > 0$, and every zero-knowledge attacker $\mathcal{A}$ against $\mathsf{DSFS}[\Sigma,\mathcal{R}]$ (with parameter r as in Fig. 4) given an arbitrarily $\mathcal{R}$-dependent advice z, and given at most $q_{\mathcal{R}}$ queries to $\mathcal{R}/\mathsf{Sim}_{\mathcal{R}}$, there is a NIZK simulator Sim, and a pair of (simulated) advice $(z_{\mathcal{A}}, z_S)$ such that

$$\begin{aligned}&\mathbf{Adv}^{\mathsf{NIZK},\mathsf{Sim}}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{A}, z, z_{\mathcal{A}}, z_S)\\ &\leq 44k\sqrt[3]{\frac{n(q_{\mathcal{R}}+k)\log|\mathcal{Z}|}{2^{\min(r,n)-1}}} + \frac{4k^2}{2^{2n-r}} + \frac{16k^2}{3\cdot 2^n} + \mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{B}) + 2^{-2n} + \delta + \zeta_\Sigma ,\end{aligned}$$

where the running time of $\mathcal{A}^{\mathsf{Sim}_{\mathsf{Prv}},\mathsf{Sim}_{\mathcal{R}}}$ is roughly $T_{\mathcal{A}} + O(q_{\mathcal{R}}^2 + n^4)$ (with the local running time of $\mathcal{A}$ denoted by $T_{\mathcal{A}}$, and the running times of $\mathcal{B}$ is roughly $O(|\mathcal{Z}|\log|\mathcal{Z}| \cdot (T_{\mathcal{A}} + q_{\mathcal{R}}(q_{\mathcal{R}}+q_w)\cdot n^4)/\delta^2)$. Moreover, in the special case where $z := \perp$ is a fixed value, and so are $(z_{\mathcal{A}}, z_S) := (\perp,\perp)$, the (sub-exponential) quantum PRP assumption can be relaxed, with

$$\mathbf{Adv}^{\mathsf{NIZK},\mathsf{Sim}}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{A}, \perp^3) \leq 44k\sqrt[3]{\frac{n(q_{\mathcal{R}}+k)\log|\mathcal{Z}|}{2^{\min(r,n)-1}}} + \frac{4k^2}{2^{2n-r}} + \frac{16k^2}{3\cdot 2^n} + 2^{-2n} + \zeta_\Sigma .$$

Proof. On a high level, the proof follows the traditional outline of proofs for Fiat-Shamir when using a random oracle: the code of Prv is replaced over several game hops. First, every call to $\mathcal{R}$ with input x is replaced by first sampling a random output y and then programming $\mathcal{R}$ to agree with $x \mapsto y$ (c.f., Trans in Fig. 6). Then, all calls to P_Σ as well as samplings that correspond to challenges are moved to the very beginning of the code such that they can be externalized into a call to $\Sigma(\mathrm{x},\mathrm{w})$ next (c.f., $\mathsf{Sim}_{\mathsf{Trans}}$ in Fig. 5). After this syntactic change, the

$\mathsf{Sim}_{\mathsf{Trans}}(\mathrm{x},\mathrm{w})$ simulates below:

1: $(a_1,...,a_k,\mathsf{ch}_1,...,\mathsf{ch}_{k-1}) \leftarrow \Sigma(\mathrm{x},\mathrm{w})$
2: $s_0 := \mathsf{P3FQ}(\mathrm{x}\|0^{2n-r})$
3: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
4: $s_{i+1} \leftarrow \{\mathsf{ch}_i\} \times \{0,1\}^{2n-r}\ \forall i \in [k-1]$
5: $\mathsf{Prog}((\tau\|0^{2n-r}) \oplus s_0, s_1)$
6: **for** $i = 1,\ldots,k-1$
7: $\quad \mathsf{Prog}((a_i\|0^{2n-r}) \oplus s_i, s_{i+1})$
8: **return** $(\tau, a_1, \ldots, a_k)$

$\mathsf{Sim}_{\mathsf{Prv}}(\mathrm{x})$ simulates below:

1: $(a_1,...,a_k,\mathsf{ch}_1,...,\mathsf{ch}_{k-1}) \leftarrow \mathsf{Sim}_{\Sigma}(\mathrm{x})$
2: $s_0 := \mathsf{P3FQ}(\mathrm{x}\|0^{2n-r})$
3: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
4: $s_{i+1} \leftarrow \{\mathsf{ch}_i\} \times \{0,1\}^{2n-r}\ \forall i \in [k-1]$
5: $\mathsf{Prog}((\tau\|0^{2n-r}) \oplus s_0, s_1)$
6: **for** $i = 1,\ldots,k-1$
7: $\quad \mathsf{Prog}((a_i\|0^{2n-r}) \oplus s_i, s_{i+1})$
8: **return** $(\tau, a_1, \ldots, a_k)$

Fig. 5. Simulations $\mathsf{Sim}_{\mathsf{Trans}}$ and $\mathsf{Sim}_{\mathsf{Prv}}$ of Trans (Fig. 6) and Prv (Fig. 4) respectively, where both $\mathsf{P3FQ}$ (Fig. 1) and Prog (Theorem 15) are simulated using the approach described in Appendix A, which comes with an additive 2^{-2n} loss.

standard HVZK game-hop follows where $\Sigma(\mathrm{x},\mathrm{w})$ is replaced by $\mathsf{Sim}_{\Sigma}(\mathrm{x})$ and we arrived at a simulation of Prv that does not need the secret key anymore (c.f., $\mathsf{Sim}_{\mathsf{Prv}}$ in Fig. 5).

In the above, we skipped over replacing $\mathcal{R}$ to $\mathsf{P3FQ}$ (with z being switched to z' correspondingly), and replacing $\mathsf{P3FQ}$ with an efficient simulation (cf. Appendix A) which happens when moving from Trans to $\mathsf{Sim}_{\mathsf{Trans}}$ (with z' being switched to $z_{\mathcal{A}}$, and Sim given an advice z_S additionally). In summary we show:

$$
\begin{aligned}
&\mathbf{Adv}^{\mathsf{NIZK},\mathsf{Sim}}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{A}, z, z_{\mathcal{A}}, z_S) = \mathsf{SD}\left(\mathcal{A}^{\mathcal{R},\mathsf{Prv}^{\mathcal{R}}}(z), \mathcal{A}^{\mathsf{Sim}_{\mathcal{R}}^{z_S},\mathsf{Sim}_{\mathsf{Prv}}^{z_S}}(z_{\mathcal{A}})\right)\\
&\le \mathsf{SD}\left(\mathcal{A}^{\mathcal{R},\mathsf{Prv}^{\mathcal{R}}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}(z')\right) + \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}, \mathcal{A}^{\mathsf{Sim}_{\mathsf{P3FQ}}^{z_S},\mathsf{Sim}_{\mathsf{Trans}}^{z_S}}\right)\\
&\quad + \mathsf{SD}\left(\mathcal{A}^{\mathsf{Sim}_{\mathsf{P3FQ}}^{z_S},\mathsf{Sim}_{\mathsf{Trans}}^{z_S}}(z_{\mathcal{A}}), \mathcal{A}^{\mathsf{Sim}_{\mathsf{P3FQ}}^{z_S},\mathsf{Sim}_{\mathsf{Prv}}^{z_S}}(z_{\mathcal{A}})\right)\\
&\le \mathsf{SD}\left(\mathcal{A}^{\mathcal{R},\mathsf{Prv}^{\mathcal{R}}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}(z')\right) + \mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{B}) + \delta + \zeta_{\Sigma}\ , \qquad (9)
\end{aligned}
$$

where the second inequality follows from using the efficient simulation of $\mathsf{P3FQ}$ laid out in Appendix A to replace the second term, and observing that the last SD is exactly the HVZK error for Σ; we also did not include the advice z' as input to $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}$ and $\mathcal{A}^{\mathsf{Sim}_{\mathsf{P3FQ}}^{z_S},\mathsf{Sim}_{\mathsf{Trans}}^{z_S}}$ above to reduce notational clutter. Here, it has to be noted that the simulation comes at the cost of a $O(|\mathcal{Z}|\log|\mathcal{Z}|\cdot(T_{\mathcal{A}} + q_{\mathcal{R}}(q_{\mathcal{R}}+q_w)\cdot n^4)/\delta^2)$ running time, where $T_{\mathcal{A}}$ is the local running time of the $\mathcal{A}$. Moreover, in the case where there is no advice (i.e. $z=\perp$), then so is $(z_{\mathcal{A}}, z_S) = (\perp,\perp)$, the $\mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{B}) + 2^{-2n} + \delta$ term can be replaced with 2^{-2n}, as explained in Appendix A.

It remains to bound the distance between $\mathcal{A}^{\mathcal{R},\mathsf{Prv}^{\mathcal{R}}}(z)$ and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}(z')$. It turns out that in the DSFS case, this step is non-trivial. Unlike a random function, programming a random permutation has side effects: at least one other value is always changed (in our case, 2^n values for a $2n$-bit permutation). Thus, we must prove that successive programmings do not interfere with each other.

We therefore decompose the proof into the following hybrid sequence:

$$\begin{aligned}\mathcal{A}^{\mathcal{R},\mathsf{Prv}^{\mathcal{R}}}(z') &= \mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}(z') \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_0}(z') \\ &= \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_1}(z') \approx \cdots \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_k}(z') = \mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}(z') \ ,\end{aligned}$$

where $\mathsf{Hyb}_0, \mathsf{Hyb}_1, \ldots, \mathsf{Hyb}_k, \mathsf{Trans}$ are as specified in Fig. 6, and the first equality follows from the fact that $\mathsf{P3FQ}$ is perfectly indistinguishable from $\mathcal{R}$.

From $\mathsf{Prv}^{\mathsf{P3FQ}}$ *to* Hyb_0. The only difference between $\mathsf{Prv}^{\mathsf{P3FQ}}$ and Hyb_0 is that $\mathsf{Prv}^{\mathsf{P3FQ}}$ computes $s_1 := \mathsf{P3FQ}((\tau \| 0^{2n-r}) \oplus s_0)$, while Hyb_0 instead performs $\mathsf{P3FQ}$ reprogramming (for Prog from Theorem 10) on the same input and freshly samples $s_1 \leftarrow \{0,1\}^{2n}$. Before this divergence, there is no Prog query being made, and through out each game, there are at most $q_{\mathcal{R}} + k$ queries to $\mathcal{R}$. Thus, substituting $q_{\mathcal{R}}$, q_w, and ϵ in Theorem 10 with $q_{\mathcal{R}} + k$, 0, and 2^{-r} yields

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_0}(z')\right) \leq 44\sqrt[3]{n(q_{\mathcal{R}}+k)\log|\mathcal{Z}|\,(2^{-r}+2^{-n})} \ . \tag{10}$$

From Hyb_0 *to* Hyb_1. The only difference between Hyb_0 and Hyb_1 is a syntactical one. Indeed, Hyb_1 is defined by Hyb_J when $J = 1$ which replaces all iterations before the Jth in the for-loop of Hyb_0 to a $\mathsf{P3FQ}$ reprogramming (again, for Prog as defined in Theorem 10). Hence, when $J = 1$, no changes are done, and so

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_0}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_1}(z')\right) = 0 \ . \tag{11}$$

Rewriting Hyb_J. Next, we do a simple rewrite of Hyb_J. Namely, in the Jth interation of the for-loop, we make it explicit that $s_{J+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_J)$ is computed via evaluating $\mathsf{P3FQ}$. This is just a syntactical rewrite, and does not cause any difference. We postpone the steps between Hyb_J and Hyb_{J+1} (for $J \in [k-1]$) to a later point, and for now continue from Hyb_k.

From Hyb_k *to* Trans. The only difference between Hyb_k and Trans is a syntactical one. Indeed, Hyb_k replaces $\mathsf{P3FQ}$ evaluations with suitable reprogramming for iterations before the kth of its for-loop. Since there are at most k such iterations, such a substitution is done in all iterations, which is what Trans does.

For the remainder of this proof, it suffices to show $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_{J+1}}$ for each $J \in [k-1]$. Inspecting Hyb_J and Hyb_{J+1}, one already sees that the only difference is the latter replaces $s_{J+1} := \mathsf{P3FQ}((a_J \| 0^{2n-r}) \oplus s_J)$ at the Jth iteration with a suitable reprogramming $\mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1})$ for $s_{J+1} \leftarrow \{0,1\}^{2n}$. Since s_J has high min-entropy, one expects this change to be unnoticeable. However, making this formal is slightly more tedious, as s_J is not "freshly" sampled — it has been used to program $\mathsf{P3FQ}$ in the previous iteration, and so our reprogramming theorem (Theorem 10) does not directly apply. Instead, we go through the following hybrid sub-sequence:

$$\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'_J} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}''_J} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'''_J} = \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_{J+1}},$$

where again the oracles $\mathsf{Hyb}'_J, \mathsf{Hyb}''_J, \mathsf{Hyb}'''_J$ are as specified in Fig. 6; we also did not include the advice z' as input to each of the games in the above sub-sequence to reduce notational clutter.

From Hyb_J *to* Hyb'_J. We consider Hyb'_J, which defers all Prog queries before the Jth iteration of the for-loop until after s_{J+1} is evaluated at the Jth iteration. Intuitively, this change only causes a difference if the deferred Prog queries "collide" with the evaluation of $s_{J+1} := \mathsf{P3FQ}((a_J \| 0^{2n-r}) \oplus s_J)$.

Formally, let R_j be the right-most n bits of $\mathsf{F}_1^{H_1} \circ \mathcal{Q}\left((a_j \| 0^{2n-r}) \oplus s_j\right)$, where F_1 denotes the 1-round Feistel as in Section 2.3, and L'_j be the left-most n bits of $\mathcal{P}^{-1}(s'_{j+1})$ where $s'_{j+1} := s_{j+1}$ for $j \in [J-1]$ and $s'_{J+1} := \mathsf{P3FQ}_0((a_j \| 0^{2n-r}) \oplus s_j)$ with $\mathsf{P3FQ}_0$ being the initial function value of $\mathsf{P3FQ}$ before any reprogramming has taken place. Then, the following (bad) event

$$\Delta := \left[\, R_J \in \{R_1, \ldots, R_{J-1}\} \text{ or } L'_J \in \{L'_1, \ldots, L'_{J-1}\} \,\right]$$

is well-defined, and happens with the same probability (which we then denote by $\Pr[I]$) in both $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}(z')$ and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'_J}(z')$ games. We consider Δ a bad event because these R_i and L'_j values are exactly where $\mathsf{P3FQ}$ gets programmed (in the second and third Feistel rounds respectively) by Prog, and so conditioned on $\neg\Delta$ (namely the event where Δ does not happen), both Hyb_J and Hyb'_J behave identically. Therefore we have $\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'_J}(z')\right) \leq \Pr[\Delta]$.

Observe now that the event $R_J \in \{R_1, \ldots, R_{J-1}\}$ is very unlikely to happen. This is because their pre-images $\left((a_j \| 0^{2n-r}) \oplus s_j\right)_{j \in [J]}$ are generated via interacting with $\mathsf{P3FQ}$ only, and hence independent of the entire function table of $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ because $\mathcal{P}$ is sampled independently. Therefore, for each $j \in [J-1]$, either the event

$$\Lambda_j : \quad \left[\, (a_j \| 0^{2n-r}) \oplus s_j = (a_J \| 0^{2n-r}) \oplus s_J \,\right]$$

happens which is of probability at most $2^{-(2n-r)}$ because

$$\begin{aligned} H_\infty(s_J \mid s_j, a_j, a_J) &= H_\infty\left(s_J \;\middle|\; s_j, a_j, \mathcal{P}_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, s_J|_{1,\ldots,r})\right) \\ &\geq H_\infty\left(s_J \;\middle|\; s_J|_{1,\ldots,r}\right) \geq 2n - r \,, \end{aligned}$$

or Λ_j doesn't happen (denoted by $\neg\Delta'_j$), conditioned on which the (marginal) distribution of (R_j, R_J) is identical to that of n-bit suffixes of two uniform samples of $2n$-bit strings without replacement, and then $R_j = R_J$ only happens with probability at most $2^n/(2^{2n}-1)$. This in turn gives us

$$\begin{aligned} \Pr\left[\, R_J \in \{R_1, \ldots, R_{J-1}\} \,\right] &\leq \sum_{j \in [J-1]} \left(\Pr\left[\, \Lambda_j \,\right] + \Pr\left[\, R_j = R_J \wedge \neg\Lambda'_j \,\right] \right) \\ &\leq (J-1)\left(2^{-(2n-r)} + \frac{2^n}{2^{2n}-1}\right) \leq \frac{k}{2^{2n-r}} + \frac{4k}{3 \cdot 2^n} \end{aligned}$$

Following the same argument, the event $L'_J \in \{L'_1, \ldots, L'_{J-1}\}$ is also very unlikely to happen — conditioned on the event $\Lambda'_j : \left[\, s'_{j+1} = s'_{J+1} \,\right]$ (of probability at most $1/2^{2n-r}$) not happening, the (marginal) distribution of (L'_j, L'_J) for each $j \in [J-1]$ is identical to that of n-bit prefixes of two uniform samples of $2n$-bit strings without replacement, and so again

$$\begin{aligned}\Pr\left[\, L'_J \in \{L'_1, \ldots, L'_{J-1}\} \,\right] &\leq \sum_{j \in [J-1]} \Big(\Pr\left[\, \Lambda'_j \,\right] + \Pr\left[\, L'_j = L'_J \wedge \neg \Lambda'_j \,\right] \Big) \\ &\leq \frac{k}{2^{2n-r}} + \frac{4k}{3 \cdot 2^n} \,.\end{aligned}$$

Putting things together, we have $\Pr[\,\Delta\,] \leq 2k/2^{2n-r} + 8k/(3 \cdot 2^n)$ and hence

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'_J}(z')\right) \leq \Pr[\,\Delta\,] \leq \frac{2k}{2^{2n-r}} + \frac{8k}{3 \cdot 2^n} \,.$$

From Hyb'_J *to* Hyb''_J. The only difference between Hyb'_J and Hyb''_J is that the former evaluates $s_{J+1} := \mathsf{P3FQ}((a_J \| 0^{2n-r}) \oplus s_J)$, while the latter programs the same point with $\mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1} \leftarrow \{0,1\}^{2n})$. Note that this is the first reprogramming throughout the execution because in prior hybrid steps, we have deferred all other programming. Therefore, immediately from substituting $q_{\mathcal{R}}, q_w$, and ϵ in Theorem 10 with $q_{\mathcal{R}} + k$, 0, and 2^{-r}, we get

$$\mathsf{SD}(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}''_J}(z')) \leq 44 \sqrt[3]{n(q_{\mathcal{R}} + k) \log |\mathcal{Z}| \, (2^{-r} + 2^{-n})} \,.$$

From Hyb''_J *to* Hyb'''_J. The only difference between Hyb''_J and Hyb'''_J is that the latter defers the reprogramming $\mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1} \leftarrow \{0,1\}^{2n})$ until after all other reprogrammings $\mathsf{Prog}((a_j \| 0^{2n-r}) \oplus s_j, s_{j+1} \leftarrow \{0,1\}^{2n})$ for $j \in [J-1]$ are performed, while the former does not. Similar to the step from Hyb_J to Hyb'_J, this deferring of reprogramming only causes a difference if the reprogramming "collides", which happens with probability at most $2k/2^{2n-r} + 8k/(3 \cdot 2^n)$. Therefore we have

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}''_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'''_J}(z')\right) \leq \frac{2k}{2^{2n-r}} + \frac{8k}{3 \cdot 2^n} \,.$$

From Hyb'''_J *to* Hyb_{J+1}. The only difference between Hyb'''_J and Hyb_{J+1} is a syntactical one. Indeed, both oracles query Prog with the same input for the transcripts generated in the first J iterations of the for-loop, and evaluate $s_{i+1} := \mathsf{P3FQ}((a_j \| 0^{2n-r}) \oplus s_i)$ for $i > J$. Therefore, we have

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}'''_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_{J+1}}(z')\right) = 0 \,.$$

Collecting the bounds thus far, we obtain

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}(z'), \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_{J+1}}(z')\right) \\ &\leq 44 \sqrt[3]{n(q_{\mathcal{R}} + k) \log |\mathcal{Z}| \, (2^{-r} + 2^{-n})} + \frac{4k}{2^{2n-r}} + \frac{16k}{3 \cdot 2^n} \,. \qquad (12)\end{aligned}$$

Hence, we obtain

$$\begin{aligned}
&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}\right) = \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_k}\right) \\
&\leq \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_0}\right) + \sum_{J\in[k-1]\cup\{0\}} \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_{J+1}}\right) \\
&\leq 44k\sqrt[3]{n(q_{\mathcal{R}}+k)\log|\mathcal{Z}|\left(2^{-r}+2^{-n}\right)} + \frac{4k^2}{2^{2n-r}} + \frac{16k^2}{3\cdot 2^n}\ ,
\end{aligned}$$

where the equality follows from the fact that Hyb_k behaves identically to Trans, and the last inequality follows from combining (10), (11), and (12); we also did not include the advice z' as input to $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Prv}^{\mathsf{P3FQ}}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}}$, and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{Hyb}_J}$ for $J \in [k]$ in the above (in)equalities to reduce notational clutter. Plugging the above back to (9), the proof is concluded.

□

Acknowledgment

Part of this work was done while Yu-Hsuan Huang was employed at CWI Amsterdam, and supported by the Dutch Research Agenda (NWA) project HAPKIDO (Project No. NWA.1215.18.002), which is financed by the Dutch Research Council (NWO). Andreas Hülsing was funded by an NWO VIDI grant (Project No. VI.Vidi.193.066). This work was done while the Varun Maram was at SandboxAQ. Silvia Ritsch is part of the Quantum-Safe Internet (QSI) ITN which received funding from the European Union's Horizon-Europe programme as Marie Sklodowska-Curie Action (PROJECT 101072637 - HORIZON -MSCA-2021-DN-01). Abishanka Saha was funded in part by the European Commission through the Horizon Europe program under project number 101135475 (TALER) and in part by the Swiss State Secretariat for Education, Research and Innovation (SERI).

References

ABK+24. Gorjan Alagic, Chen Bai, Jonathan Katz, Christian Majenz, and Patrick Struck. Post-quantum security of tweakable Even-Mansour, and applications. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 310–338. Springer, Cham, May 2024.

ABKM22. Gorjan Alagic, Chen Bai, Jonathan Katz, and Christian Majenz. Post-quantum security of the Even-Mansour cipher. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 458–487. Springer, Cham, May / June 2022.

ACMT25. Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. FOCS 2025 (To Appear), 2025.

AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 269–295. Springer, Cham, August 2019.

BDF+11. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.

BDMX25. Benjamin Bencina, Benjamin Dowling, Varun Maram, and Keita Xagawa. Post-quantum cryptographic analysis of SSH. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 595–613. IEEE Computer Society Press, May 2025.

BDPV12. Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the Sponge: Single-pass authenticated encryption and other applications. In Ali Miri and Serge Vaudenay, editors, *SAC 2011*, volume 7118 of *LNCS*, pages 320–337. Springer, Berlin, Heidelberg, August 2012.

BDPVA07. Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. Ecrypt Hash Workshop, 2007.

BHH+19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019.

BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.

BZ13. Dan Boneh and Mark Zhandry. Quantum-secure message authentication codes. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 592–608. Springer, Berlin, Heidelberg, May 2013.

CAD+20. David A. Cooper, Daniel C. Apon, Quynh H. Dang, Michael S. Davidson, Morris J. Dworkin, and Carl A. Miller. Recommendation for stateful hash-based signature schemes. Technical Report NIST Special Publication (SP) 800-208, National Institute of Standards and Technology, Gaithersburg, MD, 2020.

Car25. Joseph Carolan. Compressed permutation oracles. Cryptology ePrint Archive, Paper 2025/1734, 2025.

CFHL21. Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 598–629. Springer, Cham, October 2021.

CGLQ20. Kai-Min Chung, Siyao Guo, Qipeng Liu, and Luowen Qian. Tight quantum time-space tradeoffs for function inversion. In *61st FOCS*, pages 673–684. IEEE Computer Society Press, November 2020.

CHL+25. Alexandru Cojocaru, Minki Hhan, Qipeng Liu, Takashi Yamakawa, and Aaram Yun. Quantum lifting for invertible permutations and ideal ciphers. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part II*, volume 16001 of *LNCS*, pages 481–512. Springer, Cham, August 2025.

CMSZ19. Jan Czajkowski, Christian Majenz, Christian Schaffner, and Sebastian Zur. Quantum lazy sampling and game-playing proofs for quantum indifferentiability. Cryptology ePrint Archive, Report 2019/428, 2019.

CO25. Alessandro Chiesa and Michele Orrù. A Fiat–Shamir transformation from duplex sponges. Cryptology ePrint Archive, Paper 2025/536, 2025.

CP24. Joseph Carolan and Alexander Poremba. Quantum one-wayness of the single-round Sponge with invertible permutations. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VI*, volume 14925 of *LNCS*, pages 218–252. Springer, Cham, August 2024.

DFM20. Jelle Don, Serge Fehr, and Christian Majenz. The measure-and-reprogram technique 2.0: Multi-round fiat-shamir and more. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 602–631. Springer, Cham, August 2020.

DFMS22. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022.

DJL+24. Yevgeniy Dodis, Aayush Jain, Huijia Lin, Ji Luo, and Daniel Wichs. How to simulate random oracles with auxiliary input. In *65th FOCS*, pages 1207–1230. IEEE Computer Society Press, October 2024.

EM97. Shimon Even and Yishay Mansour. A construction of a cipher from a single pseudorandom permutation. *Journal of Cryptology*, 10(3):151–162, June 1997.

FFH25. Pouria Fallahpour, Serge Fehr, and Yu-Hsuan Huang. Tighter quantum security for Fiat-Shamir-with-aborts and hash-and-sign-with-retry signatures. Cryptology ePrint Archive, Paper 2025/985, 2025.

FH23. Serge Fehr and Yu-Hsuan Huang. On the quantum security of HAWK. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023*, pages 405–416. Springer, Cham, August 2023.

GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.

GP07. Louis Granboulan and Thomas Pornin. Perfect block ciphers with small blocks. In Alex Biryukov, editor, *FSE 2007*, volume 4593 of *LNCS*, pages 452–465. Springer, Berlin, Heidelberg, March 2007.

HBG+18. Andreas Huelsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, 2018.

Hos25. Akinori Hosoyamada. Post-quantum security of keyed sponge-based constructions through a modular approach. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part I*, volume 16245 of *Lecture Notes in Computer Science*, pages 411–445. Springer, 2025.

HRS16. Andreas Hülsing, Joost Rijneveld, and Fang Song. Mitigating multi-target attacks in hash-based signatures. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part I*, volume 9614 of *LNCS*, pages 387–416. Springer, Berlin, Heidelberg, March 2016.

HXY19. Minki Hhan, Keita Xagawa, and Takashi Yamakawa. Quantum random oracle model with auxiliary input. In Steven D. Galbraith and Shiho Mo-

riai, editors, *ASIACRYPT 2019, Part I*, volume 11921 of *LNCS*, pages 584–614. Springer, Cham, December 2019.

KM10. Hidenori Kuwakado and Masakatu Morii. Quantum distinguisher between the 3-round feistel cipher and the random permutation. In *2010 IEEE International Symposium on Information Theory*, pages 2682–2685, 2010.

LZ19. Qipeng Liu and Mark Zhandry. Revisiting post-quantum Fiat-Shamir. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 326–355. Springer, Cham, August 2019.

MMW24. Christian Majenz, Giulio Malavolta, and Michael Walter. Permutation superposition oracles for quantum query lower bounds. Cryptology ePrint Archive, Report 2024/1140, 2024.

MMW25. Christian Majenz, Giulio Malavolta, and Michael Walter. Permutation superposition oracles for quantum query lower bounds. In Michal Koucký and Nikhil Bansal, editors, *57th ACM STOC*, pages 1508–1519. ACM Press, June 2025.

Mor05. Ben Morris. The mixing time of the Thorp shuffle. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 403–412. ACM Press, May 2005.

MR14. Ben Morris and Phillip Rogaway. Sometimes-recurse shuffle - almost-random permutations in logarithmic expected time. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 311–326. Springer, Berlin, Heidelberg, May 2014.

NC10. Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

Pre17. Thomas Prest. Sharper bounds in lattice-based cryptography using the Rényi divergence. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 347–374. Springer, Cham, December 2017.

RY13. Thomas Ristenpart and Scott Yilek. The mix-and-cut shuffle: Small-domain encryption secure against N queries. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 392–409. Springer, Berlin, Heidelberg, August 2013.

SS12. Emil Stefanov and Elaine Shi. FastPRP: Fast pseudo-random permutations for small domains. Cryptology ePrint Archive, Report 2012/254, 2012.

Unr14. Dominique Unruh. Quantum position verification in the random oracle model. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2014.

Unr21. Dominique Unruh. Compressed permutation oracles (and the collision-resistance of Sponge/SHA3). Cryptology ePrint Archive, Report 2021/062, 2021.

Unr23. Dominique Unruh. Towards compressed permutation oracles. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 369–400. Springer, Singapore, December 2023.

Win99. A. Winter. Coding theorem and strong converse for quantum channels. *IEEE Transactions on Information Theory*, 45(7):2481–2485, 1999.

Zha12. Mark Zhandry. Secure identity-based encryption in the quantum random oracle model. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 758–775. Springer, Berlin, Heidelberg, August 2012.

Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.

Zha21. Mark Zhandry. Redeeming reset indifferentiability and applications to post-quantum security. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 518–548. Springer, Cham, December 2021.

Zha25. Mark Zhandry. A Note on Quantum-Secure PRPs. *Quantum*, 9:1696, April 2025.

Supplementary Material

A Efficient Simulation of P3FQ

A.1 Unconditional Simulation without Advice

In the uniform setting, where distinguishers are not given oracle-dependent advice, it is possible to efficiently simulate the P3FQ construction *unconditionally*. To do so, it suffices to simulate quantum queries to the underlying random oracles H_1, H_2, H_3, and the two random permutations $\mathcal{P}, \mathcal{Q}$. Below, we assume for simplicity that P3FQ (and hence, $\mathcal{P}$ and $\mathcal{Q}$) operates on $\{0,1\}^{2n}$, while H_1, H_2, H_3 operate on $\{0,1\}^n$, and count each integer arithmetic operation as unit-time.

Starting with read-only (non-programming) queries, each round function H_i can be simulated using a $2q$-wise independent function [Zha12] or the compressed oracle technique (see [Zha19] and [CFHL21, Appendix A]) in roughly $O(q^2)$ time, where q is the number of quantum queries to H_i.

However it is currently still open as to whether one can efficiently and perfectly simulate quantum queries to a random permutation (such as $\mathcal{P}$ or $\mathcal{Q}$). But the simulation can be done efficiently if one tolerates a vanishingly small statistical error. As treated in a line of works [Mor05,GP07,SS12,RY13,MR14], and later pointed out in [Zha25], there exists an efficient block cipher $\mathcal{C}$ in the ROM — referred to as *function-to-permutation converter (FPC)* in [Zha25] — that is indistinguishable from a random permutation, even if the distinguisher is given unbounded (and hence, quantum) query access to $\mathcal{C}$. Composing this FPC with any aforementioned simulation of random oracles would then yield a (non-perfect, yet quantum indistinguishable) simulation of a random permutation.

Note that the cost (in security loss and running time) of simulating our P3FQ construction is dominated by the cost of simulating the underlying random permutations $\mathcal{P}$ and $\mathcal{Q}$. Hence, we now specify the cost of simulating q quantum queries to a random permutation. As considered in [RY13] and later re-conceptualized in [MR14], combining the *mix-and-cut* (MC) shuffle with the *swap-or-not* (SN) cipher (which can be seen a specific instantiation of FPC) would yield a simulation with distinguishing advantage being at most 2^{-2n-1} and takes $O(q \cdot n^2)$ time when counting each random oracle query as unit-time. Further simulating the random oracle from scratch, we thus get a $O(q^2 \cdot n^4)$ time simulation using k-wise independent functions for $k \leq O(q \cdot n^2)$ or the compressed oracle technique.

Finally, to efficiently simulate programming of our P3FQ construction, it suffices to simulate programming of the underlying random oracles H_1, H_2, H_3, which can be achieved via the approach elaborated in [FFH25, Appendix A]. The running time overhead for simulating this part is $O(q_w q_{\mathcal{R}})$, for at most q_w reprogramming queries and $q_{\mathcal{R}}$ P3FQ queries.

A.2 Simulation with Advice via Complexity Leveraging

To simulate P3FQ where an attacker is given an advice z (over a finite nonempty set $\mathcal{Z}$) arbitrarily dependent on P3FQ, we first note that the joint distribution

of $(\mathcal{P}, \mathsf{P3FQ}, H_1, H_2, H_3)$ is identical to that of $(\mathcal{P}, \mathcal{R}, H_1, H_2, H_3)$, for $\mathcal{P}, \mathcal{Q}, \mathcal{R}$ sampled as uniform and independent random permutations, and for H_1, H_2, H_3 sampled as uniform and independent random functions, and so there is another $\mathcal{R}$-dependent random variable z' such that we have $(\mathcal{P}, \mathcal{R}, H_1, H_2, H_3, z')$ and $(\mathcal{P}, \mathsf{P3FQ}, H_1, H_2, H_3, z)$ to be identically distributed. It is then not too difficult to see that, for every oracle algorithm $\mathcal{A}$, there is another oracle algorithm $\mathcal{B}$ (specified below), such that

$$\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Prog}}(z) \sim \mathcal{B}^{\mathcal{P},\mathcal{P}^{-1},\mathcal{R},\mathcal{R}^{-1},H_1,H_2,H_3}(z')$$

are identically distributed, where Prog is our $\mathsf{P3FQ}$ reprogramming oracle as specified in Theorem 10.

$\mathcal{B}^{\mathcal{P},\mathcal{P}^{-1},\mathcal{R},\mathcal{R}^{-1},H_1,H_2,H_3}(z')$
1: $D_i(X) := \perp\ \forall(i, X)$
2: $b \leftarrow \mathcal{A}^{\mathcal{R}',\mathcal{R}'^{-1},\mathsf{Prog}'}(z')$
3: **return** b

$H'_i(X)$:=
1: $H_i(X)$ if $D_i(X) = \perp$
2: $D_i(X)$ otherwise
$\mathcal{Q}' := (\mathcal{P} \circ \mathsf{F}_3^{H_1,H_2,H_3})^{-1} \circ \mathcal{R}$
$\mathcal{R}' := \mathcal{P} \circ \mathsf{F}_3^{H_1,H'_2,H'_3} \circ \mathcal{Q}'$

$\mathsf{Prog}'(X, Y)$:
1: $(L, R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}'(X)$
2: $(L', R') := \mathcal{P}^{-1}(Y)$
3: $D_2(R) := L \oplus U$
4: $D_3(U) := V \oplus R$

Fig. 7. Simulation of $\mathsf{P3FQ}$ and Prog via read-only queries to $\mathcal{P}, \mathcal{R}, H_1, H_2, H_3$.

Observe that, since $(H_1, H_2, H_3, \mathcal{P}) \to \mathsf{P3FQ} \to z$ forms a Markov chain, so does $(H_1, H_2, H_3, \mathcal{P}) \to \mathcal{R} \to z'$. Moreover, since (H_1, H_2, H_3) is independent of $\mathcal{R}$, it is also independent of the pair $(\mathcal{R}, z')$. We therefore consider the simulation of each H_i and $\mathcal{P}$ unconditionally, as mentioned above, which gives us another oracle algorithm $\mathcal{C}'$ so that

$$\mathsf{SD}\left(\mathcal{B}^{\mathcal{P},\mathcal{P}^{-1},\mathcal{R},\mathcal{R}^{-1},H_1,H_2,H_3}(z'), \mathcal{C}^{\mathcal{R},\mathcal{R}^{-1}}(z')\right) \leq 2^{-2n}$$

Assuming the existence of a (sub-exponentially secure) quantum PRP (qPRP), a similar argument as in [DJL+24, Section 3] then tells us that there is a simulated pair of advices $(z_{\mathcal{C}}, z_S)$ and a simulator $\mathsf{Sim}_{\mathcal{R}}$ that is given one z_S (denoted by $\mathsf{Sim}_{\mathcal{R}}^{z_S}$), such that

$$\mathsf{SD}\left(\mathcal{C}^{\mathcal{R},\mathcal{R}^{-1}}(z'), \mathcal{C}^{\mathsf{Sim}_{\mathcal{R}}^{z_S}}(z_{\mathcal{C}})\right) \leq \mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{D}) + \delta$$

for an algorithm $\mathcal{D}$ of runtime $O(|\mathcal{Z}| \log |\mathcal{Z}| \cdot T_{\mathcal{C}}/\delta^2)$, where $T_{\mathcal{C}}$ denotes the local running time of $\mathcal{C}$, which sums up to $O(T_{\mathcal{A}} + q_{\mathcal{R}}(q_{\mathcal{R}} + q_w) \cdot n^4)$.

Putting things together, we have obtained a simulated pairs of advices $(z_{\mathcal{C}}, z_S)$, and a similarly efficient algorithm $\mathcal{C}^{\mathsf{Sim}_{\mathcal{R}}^{z_S}}$ (compared to $\mathcal{A}$), such that

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Prog}}, \mathcal{C}^{\mathsf{Sim}_{\mathcal{R}}^{z_S}}(z_{\mathcal{C}})\right) \leq \mathbf{Adv}^{\mathsf{qPRP}}(\mathcal{D}) + 2^{-2n} + \delta\ ,$$

with the running time of $\mathcal{D}$ being $O(|\mathcal{Z}| \log |\mathcal{Z}| \cdot (T_{\mathcal{A}} + q_{\mathcal{R}}(q_{\mathcal{R}} + q_w) \cdot n^4)/\delta^2)$, for which the sub-exponentially secure qPRP (with sufficiently large parameter) is still secure.

B Proof of Theorem 3

Theorem 3. *Let $n \in \mathbb{Z}_{>0}$, and $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$. Let $\mathcal{A}$ be an oracle algorithm given at most q bidirectional quantum queries to $\mathcal{R}$. Then for $X \leftarrow \mathcal{A}^{\mathcal{R},\mathcal{R}^{-1}}$, we have*

$$\Pr\left[\{X, \mathcal{R}(X)\} \subseteq \{0,1\}^n \times \{0\}^n\right] \leq \frac{18916(q+1)^3 n + 12}{2^n} .$$

Proof. Since $\mathsf{P3FQ}$ is perfectly indistinguishable from a random permutation, swapping $\mathcal{R}$ for $\mathsf{P3FQ}$ does not change the statement as in Theorem 3. We then consider a stronger setting where attackers are not only given query access to $\mathsf{P3FQ}$, but are given the entire function tables of $\mathcal{P}, \mathcal{Q}$, and H_3, and bounded query access to the random oracles H_1 and H_2 for the first two rounds.

Indeed, via inspection, any successful attacker in Theorem 3 can be turned into a successful attacker in Lemma 12 (stated below) with only a mild blow-up in query complexity. Specifically, note that each quantum query to $\mathsf{P3FQ}$ can be implemented using two quantum queries to each of H_1, H_2, and once a double-sided $X \leftarrow \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1}}$ is produced, in two additional classical queries, one can derive the internal RO queries for $\mathsf{P3FQ}(X)$. These sum up to $4q+2$ random oracle queries to H_1 and H_2 in total. Hence, it suffices to prove Lemma 12. □

Lemma 12. *Let $n \in \mathbb{Z}_{>0}$, $\mathcal{P}, \mathcal{Q}$ be independent random permutations over $\{0,1\}^{2n}$, and H_1, H_2, H_3 be independent random oracles $\{0,1\}^n \to \{0,1\}^n$. Let $\mathcal{A}$ be an attacker that is given the entire function tables of $\mathcal{P}, \mathcal{Q}$, and H_3, and at most q quantum queries to H_1 and H_2 in total. Then, we have the following for $(x_i, y_i)_{i\in[2]} \leftarrow \mathcal{A}^{H_1,H_2}(\mathcal{P}, \mathcal{Q}, H_3)$:*

$$\Pr\left[\begin{array}{c} y_1 = H_1(x_1) \ \wedge \ y_2 = H_2(x_2) \\ \mathcal{Q}^{-1}(x_1, (y_1 \oplus x_2)) \in \{0,1\}^n \times \{0\}^n \\ \mathcal{P} \circ \mathsf{F}_1^{H_3}((x_1 \oplus y_2), x_2) \in \{0,1\}^n \times \{0\}^n \end{array}\right] \leq \frac{40e^2 q^3 n + 12}{2^n} .$$

To prove Lemma 12, we briefly recall the transition capacity framework proposed in [CFHL21]. It provides a powerful way to argue query complexity bounds in the setting where an attacker is given quantum access[12] to a random oracle $H : \mathcal{S} \to \mathcal{Y}$, and tasked to produce k input-output pairs $(X_i, H(X_i))_{i\in[k]}$ that satisfy an apriori fixed relation $R \subseteq (\mathcal{S} \times \mathcal{Y})^k$.

Let q be the number of queries that an attacker $\mathcal{A}$ is allowed to make. The first part of this framework is to identify an inclusion chain of database properties $P_0 \subseteq P_1 \subseteq \cdots \subseteq P_q \in \mathfrak{D}_{\mathcal{S}}^{\mathcal{Y}}$ such that the empty database $\perp$ falls in P_0, and for every $D_q \in P_q$ there are $X_1, \ldots, X_k \in \mathsf{Dom}(D_q)$ such that $(X_i, D_q(X_i))_{i\in[k]} \in R$.

[12] [CFHL21] considers parallel queries. In each query round, an attacker can make multiple queries simultaneously. For simplicity, though, we will only consider sequential queries, without any such parallelism.

Immediately, the framework defines the *transition capacity* $[\![P_{i-1} \to P_i]\!]$ for each $i \in [q]$ so that [CFHL21, Theorem 5.7] tells us that

$$\sqrt{\Pr_{(X_i,Y_i)_{i\in[k]}\leftarrow\mathcal{A}^H}\left[\begin{array}{c} Y_i = H(X_i) \\ (X_i,Y_i)_{i\in[k]} \in R \end{array}\right]} \leq \sqrt{\frac{k}{|\mathcal{Y}|}} + \sum_{i\in[q]} [\![P_{i-1} \to \neg P_i]\!] \ . \tag{13}$$

Each of these $[\![P_{i-1} \to \neg P_i]\!]$ captures the difficulty (in worst case) that a database $D \in P_{i-1}$, after one more query, falls outside of P_i, but it is not too important how they are defined. The point is that they can be upper-bounded via the formulas provided in [CFHL21], which follow from a purely classical reasoning.

One formula that is particularly powerful in upper-bounding the transition capacity $[\![P \to Q]\!]$, for two database properties P and Q, is as follows. We first need to identify a family of database properties $\{\mathcal{L}^{x,D}\}_{(x,D)\in\mathcal{S}\times\mathfrak{D}^{\mathcal{Y}}_{\mathcal{S}}}$ that is so-called 1-*local*, i.e. for every x, D, whether or not a database D' falls in $\mathcal{L}^{x,D}$ only depends on the value $D'(x)$. Since each $\mathcal{L}^{x,D}$ is 1-local, we formally write it as a subset of $\mathcal{Y} \cup \{\bot\}$, collecting all accepted values $D'(x)$. Further, we require that for every $D \in P$ and for every $(x, y) \in \mathcal{S} \times \mathcal{Y}$ such that $D[x \mapsto y] \in Q$,[13] it must be that $D(x) \neq y \in \mathcal{L}^{x,D}$. Following the terminology of [CFHL21], we say that $\{\mathcal{L}^{x,D}\}_{x,D}$ (1-non-uniformly, weakly) recognizes the transition $P \to Q$. Then, [CFHL21, Theorem 5.23] tells us that

$$[\![P \to Q]\!] \leq \max_{(x,D)\in\mathcal{S}\times P} e\sqrt{10 \cdot \Pr_{U\leftarrow\mathcal{Y}}[\, U \in \mathcal{L}^{x,D}\,]} \ . \tag{14}$$

We are now ready to prove Lemma 12.

Proof of Lemma 12. To deploy the transition capacity framework, we treat the databases D_1, D_2 (resp. random oracles H_1, H_2) as part of a bigger database $D : [3] \times \{0,1\}^n \to \{0,1\}^n \cup \{\bot\}$ (resp. $H : [2] \times \{0,1\}^n \to \{0,1\}^n$), where the restriction at $\{j\} \times \{0,1\}^n$ coincides with D_j (resp. H_j) for each j. To begin with, we define the following database properties

$$\mathsf{SZ}_i := \left\{(D_1, D_2, D_3) \;\middle|\; |D_1| + |D_2| \leq i\right\} \ ,$$

$$\mathsf{DZR}[\mathcal{P}, \mathcal{Q}, H_3] = \left\{(D_1, D_2) \;\middle|\; \begin{array}{c} \exists (x_1, y_1) \in D_1 \ \wedge \ (x_2, y_2) \in D_2 \text{ s.t.} \\ \mathcal{Q}^{-1}(x_1, (y_1 \oplus x_2)) \in \{0,1\}^n \times \{0\}^n \\ \mathcal{P} \circ \mathsf{F}_1^{H_3}((x_1 \oplus y_2), x_2) \in \{0,1\}^n \times \{0\}^n \end{array}\right\}$$

where, intuitively, the property SZ_i puts a constraint on the database sizes, and $\mathsf{DZR}[\mathcal{P}, \mathcal{Q}, H_3]$ is the (unlikely) property that $\mathsf{P3FQ}^{D_1,D_2,H_3}$ contains a double-sided zero. Indeed, for every fixed choice of $\mathcal{P}, \mathcal{Q}$, and H_3, denoted by $\mathcal{P}^\circ, \mathcal{Q}^\circ$,

[13] The notation $D[x \mapsto y]$ denotes the database which is guaranteed to coincide with D everywhere except point x, which it maps to y.

and H_3° in the following, we have

$$\sqrt{\Pr\left[\begin{array}{c} y_1 = H_1(x_1) \ \wedge \ y_2 = H_2(x_2) \\ \mathcal{Q}^{-1}(x_1, (y_1 \oplus x_2)) \in \{0,1\}^r \times \{0\}^c \\ \mathcal{P} \circ \mathsf{F}_1^{H_3}((x_1 \oplus y_2), x_2) \in \{0,1\}^r \times \{0\}^c \end{array} \middle| (\mathcal{P}, \mathcal{Q}, H_3) = (\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ)\right]}$$

$$\leq \sqrt{\frac{2}{2^n}} + \sum_{i \in [q]} [\![\mathsf{SZ}_{i-1} \setminus \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \to \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \cup \neg \mathsf{SZ}_i]\!]$$

$$\leq \sqrt{\frac{2}{2^n}} + \sum_{i \in [q]} [\![\mathsf{SZ}_{i-1} \to \neg \mathsf{SZ}_i]\!] + \sum_{i \in [q]} [\![\mathsf{SZ}_{i-1} \setminus \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \to \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]]\!]$$

$$\leq \sqrt{\frac{2}{2^n}} + 0 + q \cdot [\![\mathsf{SZ}_q \setminus \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \to \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]]\!] \ ,$$

where the first inequality is via (13), the second inequality is via the basic properties of transition capacity (see Section 5.3 of [CFHL21]), and the third inequality is via applying (14) for $\mathcal{L}^{x,D} := \emptyset$, $P := \mathsf{SZ}_{i-1}$ and $Q := \neg \mathsf{SZ}_i$.

To control the last transition capacity, for every $R, R' \in \{0,1\}^n$, define

$$S^{\mathcal{P},H_3}(R) := \left\{ L \in \{0,1\}^n \mid \mathcal{P} \circ \mathsf{F}_1^{H_3}(L, R) \in \{0,1\}^n \times \{0\}^n \right\}$$
$$S_{\mathcal{Q}}(R') := \left\{ L' \in \{0,1\}^n \mid \mathcal{Q}^{-1}(L', R') \in \{0,1\}^n \times \{0\}^n \right\} \ .$$

Using Lemma 13 presented below, we establish that the quantities $\max_R \left|S^{\mathcal{P},H_3}(R)\right|$ and $\max_{R'} |S_{\mathcal{Q}}(R')|$ are at most $2n$ except with probability

$$\Pr\left[\begin{array}{l} \max\limits_{R} \left|S^{\mathcal{P},H_3}(R)\right| > 2n \ , \text{ or} \\ \max\limits_{R'} |S_{\mathcal{Q}}(R')| > 2n \end{array}\right] \leq \frac{2 \cdot 2^n}{(1 - 2n \cdot 2^{-2n})^{2n} \cdot (2n)!} \leq \frac{8}{2^n} \ .$$

In the following, we thus consider fixed choices $\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ$ of $\mathcal{P}, \mathcal{Q}$, and H_3 satisfying such a property.

Lemma 13. *Let $n, k \in \mathbb{Z}_{>0}$, $\mathcal{P}$ be a random permutation over $\{0,1\}^{2n}$, and for every $R \in \{0,1\}^n$, let $S(R) := \{L \in \{0,1\}^n \mid \mathcal{P}(L, R) \in \{0,1\}^n \times \{0\}^n\}$ be the set of n-bit strings such that the last n bits of $\mathcal{P}(L, R)$ are all zeros. Then*

$$\Pr\left[\max_{R \in \{0,1\}^n} |S(R)| > k\right] \leq \frac{2^n}{(1 - k \cdot 2^{-2n})^k \cdot k!} \ .$$

Now consider the following local properties for every $D_1, D_2 \in \mathsf{SZ}_q \setminus \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]$, and for every $x \in \{0,1\}^n$.

$$\mathcal{L}^{(1,x),D_1,D_2} := \left\{ y_1 \in \{0,1\}^n \ \middle| \ \begin{array}{c} \exists x_2 \in \mathsf{Dom}(D_2) \text{ s.t.} \\ \mathcal{Q}^{\circ -1}(x \, , \, (y_1 \oplus x_2)) \in \{0,1\}^n \times \{0^n\} \end{array} \right\}$$

$$\mathcal{L}^{(2,x),D_1,D_2} := \left\{ y_2 \in \{0,1\}^n \ \middle| \ \begin{array}{c} \exists x_1 \in \mathsf{Dom}(D_1) \text{ s.t.} \\ \mathcal{P}^\circ \circ \mathsf{F}_1^{H_3^\circ}((x_1 \oplus y_2) \, , \, x) \in \{0,1\}^n \times \{0^n\} \end{array} \right\} \ .$$

Observe that in the definition of $\mathcal{L}^{(1,x),D_1,D_2}$, there are at most q choices for x_2, and for any choice of x_2, the number of $y_1 \in \{0,1\}^n$ such that $\mathcal{Q}^{\circ -1}(x\,,\,(y_1 \oplus x_2)) \in \{0,1\}^n \times \{0^n\}$, is bounded by $2n$. Hence we have $|\mathcal{L}^{(1,x),D_1,D_2}| \leq 2qn$. Similarly, we have $|\mathcal{L}^{(2,x),D_1,D_2}| \leq 2qn$. Moreover, since $(D_1, D_2) \notin \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]$, the following holds.

$$(D_1[x \mapsto y], D_2) \in \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ] \implies y \in \mathcal{L}^{(1,x),D_1,D_2}$$
$$(D_1, D_2[x \mapsto y]) \in \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ] \implies y \in \mathcal{L}^{(2,x),D_1,D_2}$$

In the terminology of [CFHL21], these 1-local properties $\{\mathcal{L}^{(j,x),D_1,D_2}\}$ (1-non-uniformly, weakly) recognize the transition $\mathsf{SZ}_q \backslash \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \to \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]$. Therefore, by applying (14), we have

$$\begin{aligned}
&[\![\mathsf{SZ}_q \setminus \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ] \to \mathsf{DZR}[\mathcal{P}^\circ, \mathcal{Q}^\circ, H_3^\circ]]\!] \\
&\leq \max_{\substack{(j,x)\in[2]\times\{0,1\}^n \\ (D_1,D_2)\in\mathsf{SZ}_q\setminus\mathsf{DZR}[\mathcal{P},\mathcal{Q}]}} e\sqrt{10 \Pr_{U\leftarrow\{0,1\}^n}\left[\, U \in \mathcal{L}^{(j,x,D_1,D_2)} \,\right]} \\
&= \max_{\substack{(j,x)\in[2]\times\{0,1\}^n \\ (D_1,D_2)\in\mathsf{SZ}_q\setminus\mathsf{DZR}[\mathcal{P},\mathcal{Q}]}} e\sqrt{10\frac{\left|\mathcal{L}^{(j,x,D_1,D_2)}\right|}{2^n}} \leq e\sqrt{\frac{10\cdot 2qn}{2^n}}\,.
\end{aligned}$$

Collecting the bounds so far, we obtain our desired bound in Lemma 12. All that remains now is to prove Lemma 13. □

Proof of Lemma 13. Let $\mathfrak{S}_k$ be the family of all size-k subsets of $\{0,1\}^n$, i.e.

$$\mathfrak{S}_k := \left\{ S' \subseteq \{0,1\}^n \;\middle|\; |S'| = k \right\}\,.$$

For every fixed choice of $R \in \{0,1\}^n$, we have

$$\begin{aligned}
\Pr\left[\,|S(R)| > k\,\right] &\leq \sum_{S'\in\mathfrak{S}_k} \Pr\left[\,\forall L \in S' :\; \mathcal{P}(L,R) \in \{0,1\}^n \times \{0\}^n\,\right] \\
&\leq \binom{2^n}{k} \cdot \left(\frac{2^n}{2^{2n}-k}\right)^k \leq \frac{1}{(1-k\cdot 2^{-2n})^k \cdot k!}\,,
\end{aligned}$$

where the second inequality is via $|\mathfrak{S}_k| \leq \binom{2^n}{k}$ and a straightforward calculation of the probability. Therefore, via a simple union bound, we have

$$\Pr\left[\max_{R\in\{0,1\}^n} |S(R)| > k\right] \leq \sum_{R\in\{0,1\}^n} \Pr\left[\,|S(R)| > k\,\right] \leq \frac{2^n}{(1-k\cdot 2^{-2n})^k \cdot k!}\,,$$

which concludes the proof. □

C Adaptive Reprogramming without Advice

In Section 4, we proved adaptive reprogramming theorems in a QRPM setting where the attacker is given an additional oracle-dependent advice. In this section,

we provide a correspondingly tighter bound when the attacker does not have access to such an advice. Specifically, this bound's only premise is an upper-bound on the number of permutation queries prior to the reprogramming.

We start with reprogramming in the QROM, which is a variant of [FFH25, Lemma 1] and [GHHM21, Theorem 1].

Lemma 14. *Let $\mathcal{X}$ and $\mathcal{Y}$ be finite non-empty sets, $H : \mathcal{X} \to \mathcal{Y}$ be a random oracle, and $\mathcal{A}^{H,O_b,\mathsf{Prog}}$ be an oracle algorithm given quantum query access to H, and classical query access to O_b and Prog (for $b \in \{0,1\}$) as specified below.*

$O_0(\mathcal{D})$:	$O_1(\mathcal{D})$:	$\mathsf{Prog}(X,Y)$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $H(X) := Y$
2: $Y := H(X)$	2: $H(X) := Y \leftarrow \mathcal{Y}$	
3: **return** (X,Y)	3: **return** (X,Y)	

Furthermore, $\mathcal{A}$ is only allowed to query O_b at most once, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X\leftarrow\mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$ for some (small but fixed) $\epsilon > 0$. Before the O_b query, $\mathcal{A}$ is allowed to make at most q_H and q_w queries to H and Prog respectively, while after the O_b query, $\mathcal{A}$ can make unbounded many queries to both. Then, we have

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0,\mathsf{Prog}}, \mathcal{A}^{H,O_1,\mathsf{Prog}}\right) \leq \left(2q_w + \frac{q_H}{2}\right)\epsilon + \sqrt{q_H\epsilon}.$$

Proof. We note that, when $q_w = 0$, this statement is a direct consequence of [GHHM21, Theorem 1]. In the general case, the same argument as in [FFH25, Lemma 1] generalizes line-by-line, as explained below.

Specifically, consider an algorithm $\mathcal{B}^{H,O_b}$ that simulates $\mathcal{A}^{H,O_b,\mathsf{Prog}}$, except that it book-keeps all the Prog queries locally. Note that $\mathcal{B}$ does not make any Prog query, and makes at most q_H queries to H, so $\mathsf{SD}(\mathcal{B}^{H,O_0}, \mathcal{B}^{H,O_b}) \leq q_H\epsilon/2 + \sqrt{q_H\epsilon}$. Moreover, the two algorithms $\mathcal{B}^{H,O_b}$ and $\mathcal{A}^{H,O_b,\mathsf{Prog}}$ behave identically unless, in the O_b query, the sampled point X coincides with the Prog made by $\mathcal{A}$, which happens with probability at most $q_w\epsilon$. Hence $\mathsf{SD}(\mathcal{A}^{H,O_b,\mathsf{Prog}}, \mathcal{B}^{H,O_b}) \leq q_w\epsilon$, and so

$$\begin{aligned}\mathsf{SD}\left(\mathcal{A}^{H,O_0,\mathsf{Prog}}, \mathcal{A}^{H,O_1,\mathsf{Prog}}\right) &\leq \mathsf{SD}\left(\mathcal{B}^{H,O_0}, \mathcal{B}^{H,O_1}\right) + \sum_{b\in\{0,1\}} \mathsf{SD}\left(\mathcal{A}^{H,O_b,\mathsf{Prog}}, \mathcal{B}^{H,O_b}\right) \\ &\leq q_H\epsilon/2 + \sqrt{q_H\epsilon} + 2q_w\epsilon\,,\end{aligned}$$

which concludes the proof. □

C.1 Adaptive Reprogramming Theorem

We are now ready to formally state and prove the adaptive reprogramming theorem for our P3FQ Feistel tool in the uniform setting.

Theorem 15. *Let $\mathcal{X} := \{0,1\}^n$ be a set of fixed-length strings, $\mathcal{P}$, $\mathcal{Q}$ be independent random permutations over $\mathcal{X}^2$, $H = (H_1, H_2, H_3)$ be such that $H_i : \mathcal{X} \to \mathcal{X}$ is an independently sampled random function for each $i \in [3]$, and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_b,\mathsf{Prog}}$ be an oracle algorithm given quantum query access to $\mathsf{P3FQ}, \mathsf{P3FQ}^{-1}$, and classical query access to O_b, Prog (for $b \in \{0,1\}$) as specified below.*

$O_0(\mathcal{D})$:	$O_1(\mathcal{D})$:	$\mathsf{Prog}(X,Y)$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $(L,R) := \mathcal{Q}(X)$
2: $Y := \mathsf{P3FQ}^H(X)$	2: $Y \leftarrow \mathcal{X}^2$	2: $(L',R') := \mathcal{P}^{-1}(Y)$
3: **return** (X,Y)	3: $\mathsf{Prog}(X,Y)$	3: $H_2(R \oplus H_1(L)) := L' \oplus L$
	4: **return** (X,Y)	4: $H_3(L') := R' \oplus R \oplus H_1(L)$

Furthermore, $\mathcal{A}$ is only allowed to query O_b at most once, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}^2$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X\leftarrow\mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \le \epsilon$ for some (small but fixed) $\epsilon > 0$; specifically, $\mathcal{D}$ can be described as a circuit that cannot make any queries to $\mathsf{P3FQ}$, $\mathsf{P3FQ}^{-1}$, or its internal functions. Before the O_b query, $\mathcal{A}$ is allowed to make at most $q_{\mathcal{R}}$ queries to $\mathsf{P3FQ}$ and $\mathsf{P3FQ}^{-1}$ in total, and at most q_w queries to Prog, while after the O_b query, $\mathcal{A}$ can make unbounded many queries to them. Then, we have

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}\right)\\ &\le 14\sqrt{nq_{\mathcal{R}}(\epsilon + 2^{-n})} + 48nq_w(\epsilon + 2^{-n})\,. \end{aligned} \tag{15}$$

Proof. Our proof proceeds through the following simple hybrid sequence

$$\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}} \approx \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}\,,$$

with Hyb as defined in Fig. 8, where we also rewrite O_b to a form that is more compatible with the proof. The rewriting of O_1 is done via a simple change of variables, where, instead of sampling Y uniformly and programming via $\mathsf{Prog}(X,Y)$, we uniformly sample the values y_2 and y_3 that is programmed to the last two rounds H_2 and H_3 respectively.

rewritten $O_0(\mathcal{D})$:	$\mathsf{Hyb}(\mathcal{D})$:	rewritten $O_1(\mathcal{D})$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$
2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$	2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$	2: $(L,R) := \mathsf{F}_1^{H_1} \circ \mathcal{Q}(X)$
3: $y_2 := H_2(R)$	3: $H_2(R) := y_2 \leftarrow \mathcal{X}$	3: $H_2(R) := y_2 \leftarrow \{0,1\}^n$
4: $x_3 := L \oplus y_2$	4: $x_3 := L \oplus y_2$	4: $x_3 := L \oplus y_2$
5: $y_3 := H_3(x_3)$	5: $y_3 := H_3(x_3)$	5: $H_3(x_3) := y_3 \leftarrow \mathcal{X}$
6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$	6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$	6: **return** $\mathcal{P}(x_3, y_3 \oplus R)$

Fig. 8. A simple hybrid sequence to reprogram $\mathsf{P3FQ}$.

From O_0 to Hyb. The only difference between O_0 and Hyb is that, the former evaluates the second round $y_2 := H_2(R)$, while the latter programs the second

round $H_2(R) := y_2 \leftarrow \mathcal{X}$ uniformly. Moreover, given $\mathcal{P}, \mathcal{Q}, H_1, H_3$, each $\mathsf{P3FQ}$ query can be simulated using 2 queries to H_2, and each Prog query can be simulated via programming H_2 once. Therefore, substituting $\mathcal{D}$ in Lemma 14 with the distribution described by $\mathcal{D}$ and $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ here, that samples L as mentioned above, we obtain

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}\right)\\ &\leq (2q_w + q_{\mathcal{R}})\,\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q}) + \sqrt{2q_{\mathcal{R}}\,\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q})}\,.\end{aligned}$$

Observe that $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ is independent of $\mathcal{D}$, so that conditioned on every fixed choice of $\mathcal{D}$, the distribution of the permutation $\mathsf{F}_1^{H_1} \circ \mathcal{Q}$ remains uniform. Therefore, we obtain

$$\begin{aligned}\mathsf{guess}(R \mid \mathcal{D}, \mathsf{F}_1^{H_1} \circ \mathcal{Q}) &= \mathop{\mathbb{E}}_{\mathcal{D}^\circ \sim \mathcal{D}}\left[\mathsf{guess}(R \mid \mathcal{D} = \mathcal{D}^\circ, \mathsf{F}_1^{H_1} \circ \mathcal{Q})\right]\\ &\leq \mathop{\mathbb{E}}_{\mathcal{D}^\circ \sim \mathcal{D}}\left[16\sqrt{2}n \cdot \max(\mathsf{guess}(X \mid \mathcal{D} = \mathcal{D}^\circ), 2^{-n})\right]\\ &= 16\sqrt{2}n \cdot \left(\mathsf{guess}(X \mid \mathcal{D}) + 2^{-n}\right) \leq 16\sqrt{2}n \cdot \left(\epsilon + 2^{-n}\right)\,,\end{aligned}$$

where the first (non-equality) inequality follows Lemma 9. Plugging it back in the above, we obtain

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}\right)\\ &\leq 16\sqrt{2}n \cdot (2q_w + q_{\mathcal{R}})\left(\epsilon + 2^{-n}\right) + \sqrt{32\sqrt{2}q_{\mathcal{R}}n \cdot (\epsilon + 2^{-n})}\,.\end{aligned}$$

From Hyb *to* O_1. The only difference between Hyb and O_1 is that, the former evaluates the third round $y_3 := H_3(x_3)$, while the latter reprograms the values $H_3(x_3) := y_3 \leftarrow \mathcal{X}$. Given that $x_3 := L \oplus y_2$ is chosen with y_2 being a freshly sampled value over $\mathcal{X} = \{0,1\}^n$, Lemma 14 tells us that

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}\right) \leq (2q_w + q_{\mathcal{R}}) \cdot 2^{-n} + \sqrt{2q_{\mathcal{R}} \cdot 2^{-n}}\,.$$

Collecting and simplifying the bounds so far, we conclude the proof.

□

D Proof of Lemma 6

Lemma 6. *Let $H : \mathcal{X} \to \mathcal{Y}$ be a random oracle, and $\mathcal{A}$ and $\mathcal{B}$ be oracle algorithms making at most q_H and $q_{\mathcal{B}}$ quantum queries to H respectively. Additionally, $\mathcal{A}$ is allowed to query the oracle O_b (for $b \in \{0,1\}$, as defined in Theorem 5) at most once, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X \leftarrow \mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$ for some (small but fixed) $\epsilon > 0$. Then, we have*

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0}, \mathcal{A}^{H,O_1} \;\middle|\; 1 \leftarrow \mathcal{B}^H\right) \leq 2\sqrt{(q_H + q_{\mathcal{B}})\epsilon}\,.$$

Proof. In the following, we assume the reader is familiar with Zhandry's compressed oracle technique [Zha19]. First of all, it would be convenient to think of the game below where $\mathcal{B}^H$ is first run, and then $\mathcal{A}^{H,O_b}$ is run.

Hyb_0^b:

1: **repeat**
2: let H be an RO
3: $b_{\mathcal{B}} \leftarrow \mathcal{B}^H$
4: **until** $b_{\mathcal{B}} = 1$
5: $b_{\mathcal{A}} \leftarrow \mathcal{A}^{H,O_b}$
6: **return** $b_{\mathcal{A}}$

Note that the output distribution of Hyb_0^b is identical to that of $\mathcal{A}^{H,O_b}$ conditioned on $1 \leftarrow \mathcal{B}^H$, so we have

$$\mathsf{SD}\left(\mathcal{A}^{H,O_0}, \mathcal{A}^{H,O_1} \;\middle|\; 1 \leftarrow \mathcal{B}^H\right) = \mathsf{SD}\left(\mathsf{Hyb}_0^0, \mathsf{Hyb}_0^1\right) \ .$$

Note that the only difference between O_0 and O_1 is that the former evaluates $Y := H(X)$, while the latter reprograms it via $H(X) := Y \leftarrow \mathcal{Y}$.

We then consider the game Hyb_1^b that differs from Hyb_0^b, where the H queries are simulated via compressed oracle cO on a designated register D (which is initially set to the empty database $|\perp\rangle$), and right after $X \leftarrow \mathcal{D}$ is sampled in the O_b query, the compressed oracle is decompressed and fully measured in the computational basis in order to obtain H that is used afterward. This is a perfect simulation, so $\mathsf{SD}(\mathsf{Hyb}_0^{O_b}, \mathsf{Hyb}_1^b) = 0$.

Next, we consider the game Hyb_2^b that differs from Hyb_1^b as follows: right before the full measurement, it performs another binary projective measurement $b_{\mathcal{M}} \leftarrow \mathcal{M} := \{\Pi_0 := |\perp\rangle\langle\perp|_{\mathsf{D}(X)}, \Pi_1 := I - \Pi_0\}$ on the sub-register $\mathsf{D}(X)$ where the value $H(X)$ is stored, to see if the database has recorded X (where $b_{\mathcal{M}}$ is the outcome corresponds to $\Pi_{b_{\mathcal{M}}}$). If $b_{\mathcal{M}} = 1$, then the entire game aborts. Since $b_{\mathcal{M}} = 1$ happens with the same probability in both games, it then follows from the well-known *Gentle-Measurement Lemma* [Win99] that

$$\mathsf{SD}(\mathsf{Hyb}_1^b, \mathsf{Hyb}_2^b) \leq \sqrt{\Pr\left[\mathsf{Hyb}_2^b \textbf{ aborts}\right]} \ .$$

Toward bounding the abort probability, observe that if we perform a hypothetical measurement $D \leftarrow \mathcal{M}_{\mathsf{hyp}}$ in the computational basis on D, right before $\mathcal{M}$, then the probability $\Pr[\, D(X) \neq \perp \,]$ is the same as the abort probability because $\mathcal{M}$ and $\mathcal{M}_{\mathsf{hyp}}$ commute. Moreover, by construction $|D| = |\{X^\circ \mid D(X^\circ) \neq \perp\}| \leq q_{\mathcal{B}} + q_H$ with certainty, and since $\mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$, it follows that

$$\begin{aligned}
&\Pr\left[\mathsf{Hyb}_2^b \textbf{ aborts}\right] = \Pr[\, D(X) \neq \perp \,] \\
&\leq (q_{\mathcal{B}} + q_H) \cdot \mathop{\mathbb{E}}_{\mathcal{D}^\circ \sim \mathcal{D}} [\, \mathsf{guess}(X \mid \mathcal{D}^\circ) \,] \\
&\leq (q_{\mathcal{B}} + q_H) \cdot \mathsf{guess}(X \mid \mathcal{D}) \leq (q_H + q_{\mathcal{B}})\epsilon \ .
\end{aligned}$$

Finally, we note that the abort probability in Hyb_2^0 and Hyb_2^1 is the same, and moreover, conditioned on non-abort, the value $H(X)$ is a freshly uniform random value, for which the reprogramming does not make any difference to the final output distribution. Therefore $\mathsf{SD}(\mathsf{Hyb}_2^0, \mathsf{Hyb}_2^1) = 0$. Collecting the bounds so far, the proof is concluded with the following chain of closeness (measured in statistical distance)

$$\begin{aligned}
\mathcal{A}^{H,O_0}|_{1\leftarrow\mathcal{B}^H} &\overset{0}{\approx} \mathsf{Hyb}_0^0 \overset{0}{\approx} \mathsf{Hyb}_1^0 \overset{\sqrt{(q_H+q_\mathcal{B})\epsilon}}{\approx} \mathsf{Hyb}_2^0 \\
&\wr 0 \\
\mathcal{A}^{H,O_1}|_{1\leftarrow\mathcal{B}^H} &\overset{0}{\approx} \mathsf{Hyb}_0^1 \overset{0}{\approx} \mathsf{Hyb}_1^1 \overset{\sqrt{(q_H+q_\mathcal{B})\epsilon}}{\approx} \mathsf{Hyb}_2^1 .
\end{aligned}$$

□

E Proof of Lemmas 7 and 8

Lemma 7. *Let $X_1, \ldots, X_k \in \{0,1\}^{2n}$ be all distinct, $\mathcal{R}$ be a random permutation over $\{0,1\}^{2n}$, and $\mathcal{R}_\ell(\cdot)$ be the $\mathcal{R}$-dependent function that outputs the first ℓ bits of $\mathcal{R}(\cdot)$. Then*

$$\Pr\left[\, \mathcal{R}_\ell(X_1) = \cdots = \mathcal{R}_\ell(X_k) \,\right] \leq 2^{-\ell(k-1)} \cdot \frac{1}{(1 - k \cdot 2^{-2n})^{k-1}} \,.$$

Proof. For notational simplicity, we denote $Y_i = \mathcal{R}_\ell(X_i)$ for $i \in [k]$. Now,

$$\begin{aligned}
&\Pr\left[\, \mathcal{R}_\ell(X_1) = \cdots = \mathcal{R}_\ell(X_k) \,\right] = \sum_{s\in\{0,1\}^\ell} \Pr\left[\, Y_1 = \cdots = Y_k = s \,\right] \\
&= \sum_{s\in\{0,1\}^\ell} \Pr[Y_1 = s] \cdot \Pr[Y_2 = s \,|\, Y_1 = s] \cdot \ldots \cdot \Pr[Y_k = s \,|\, \wedge_{i\in[k-1]}\, Y_i = s] \\
&= \sum_{s\in\{0,1\}^\ell} \Big(\frac{2^{2n-\ell}}{2^{2n}}\Big)\Big(\frac{2^{2n-\ell}-1}{2^{2n}-1}\Big) \cdots \Big(\frac{2^{2n-\ell}-k+1}{2^{2n}-k+1}\Big) \leq \frac{(2^{-\ell})^{k-1}}{(1 - k \cdot 2^{-2n})^{k-1}},
\end{aligned}$$

where we used the fact that for $i \in [k-1]$, $\frac{2^{2n-\ell}-i}{2^{2n}-i} \leq \frac{2^{2n-\ell}}{2^{2n}-k}$. □

Lemma 8. *Let $\epsilon \in \mathbb{R}_{>0}$ and $X_1, \ldots, X_\alpha \in \mathcal{X}$, for a finite set $\mathcal{X}$, be mutually independent random variables such that $\mathsf{guess}(X_i) \leq \epsilon$ for each $k \in [\alpha]$. Then, for every $k \in [\alpha]$, we have*

$$\Pr\Big[\, |\{X_1, \ldots, X_\alpha\}| = k \Big] \leq \left\{ {\alpha \atop k} \right\} \cdot \epsilon^{\alpha-k} \,,$$

where $\left\{ {\alpha \atop k} \right\}$, called the Stirling number of second type*, denotes the number of ways the set $[\alpha]$ can be partitioned in k non-empty subsets.*

Proof. If $|\{X_1, \ldots, X_\alpha\}| = k$, then there exists a *partition* of the set $[\alpha]$ into k mutually disjoint non-empty subsets $S \coloneqq (S_i)_{i\in[k]}$ such that for any two X_a, X_b where $a, b \in S_j$, we have $X_a = X_b$. For any such set S_j, it is not hard to see that the probability of this "collision" event — let us call it Z_{S_j} — happening among X_a's for $a \in S_j$ is bounded by $\epsilon^{|S_j|-1}$.

Hence, by summing over all such possible partitions S, we get

$$\Pr\Big[\,|\{X_1, \ldots, X_\alpha\}| = k\Big] = \sum_S \Big(\prod_{j\in[k]} \Pr[Z_{S_j}]\Big) \le \left\{\begin{matrix}\alpha\\ k\end{matrix}\right\} \cdot \epsilon^{\alpha-k}.$$

□

F Bidirectional Reprogramming with Advice

We generalize our Feistel reprogramming theorem with advice to both directions. To start with, observe that over the randomness of $\mathcal{P}, \mathcal{Q}, H_1, H_2, H_3$ within the $\mathsf{P3FQ}$ construction, the joint distribution of $\left(\mathsf{F}_1^{H_1} \circ \mathcal{Q}, H_2, H_3, \mathcal{P}\right)$ is identical to that of $\left(\mathcal{P}^{-1}, H_3, H_2, \mathcal{Q}^{-1} \circ \mathsf{F}_1^{H_1}\right)$, where F_1 denotes the one-round Feistel construction as in Section 2.3. Hence, an oracle algorithm cannot distinguish whether it is (bidirectionally) querying $\mathsf{P3FQ} = \mathcal{P} \circ \mathsf{F}_1^{H_3} \circ \mathsf{F}_1^{H_2} \circ \mathsf{F}_1^{H_1} \circ \mathcal{Q}$ and programs $\mathsf{P3FQ}(X)$ to a value of Y via querying $\mathsf{Prog}(X, Y)$ in Theorem 10, or whether it is (bidirectionally) querying $\mathsf{P3FQ}^{-1} = \mathcal{Q}^{-1} \circ \mathsf{F}_1^{H_1} \circ \mathsf{F}_1^{H_2} \circ \mathsf{F}_1^{H_3} \circ \mathcal{P}^{-1}$ and programs $\mathsf{P3FQ}^{-1}(X)$ to Y via querying $\mathsf{Prog}(Y, X)$ in Theorem 10. This extends Theorem 10 to both forward-direction reprogramming ($\mathsf{P3FQ}$) and backward-direction reprogramming ($\mathsf{P3FQ}^{-1}$), when the choice of which direction is fixed apriori. (Also note that the advice z for $\mathsf{P3FQ}$ and $\mathsf{P3FQ}^{-1}$ are identical.)

Moreover, below we show via a hybrid argument that the above can be further extended to the setting where the direction of reprogramming can be *adaptively* chosen by the attacker, and there are *multiple* points being reprogrammed.

Theorem 16. *Let $\mathcal{X} := \{0,1\}^n$ be a set of fixed-length strings, $\mathcal{Z}$ be a finite non-empty set with $|\mathcal{Z}| \ge 2$, $\mathcal{P}$ and $\mathcal{Q}$ be independent random permutations over $\mathcal{X}^2$, $H = (H_1, H_2, H_3)$ be so that each $H_i : \mathcal{X} \to \mathcal{X}$ is an independently sampled random function, z be any $\mathsf{P3FQ}$-dependent random variable over $\mathcal{Z}$ (i.e. such that $(\mathcal{P}, \mathcal{Q}, H) \to \mathsf{P3FQ} \to z$ forms a Markov chain), and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_b,\mathsf{Prog}}$ be an oracle algorithm given (bidirectional) quantum query access to $\mathsf{P3FQ}$, and classical query access to O_b and Prog (for $b \in \{0,1\}$) as specified below.*

$O_0(s \in \{\pm 1\}, \mathcal{D})$:	$O_1(s \in \{\pm 1\}, \mathcal{D})$:	$\mathsf{Prog}(X, Y)$:
1: $X \leftarrow \mathcal{D}$	1: $X \leftarrow \mathcal{D}$	1: $(L, R) \coloneqq \mathcal{Q}(X)$
2: $Y := \left(\mathsf{P3FQ}^H\right)^s(X)$	2: $Y \leftarrow \mathcal{X}^2$	2: $(L', R') \coloneqq \mathcal{P}^{-1}(Y)$
3: **return** (X, Y)	3: **if** $s = 1$ $\mathsf{Prog}(X, Y)$	3: $H_2(R \oplus H_1(L)) \coloneqq L' \oplus L$
	4: **else** $\mathsf{Prog}(Y, X)$	4: $H_3(L') \coloneqq R' \oplus R \oplus H_1(L)$
	5: **return** (X, Y)	

Furthermore, $\mathcal{A}$ makes at most $q_{\mathcal{R}}$ queries to $\mathsf{P3FQ}$ *and* $\mathsf{P3FQ}^{-1}$ *in total, at most q_w queries to* Prog*, and is only allowed to query O_b at most q_P times, with (the description of) a distribution $\mathcal{D}$ over $\mathcal{X}$ that satisfies $\mathbb{E}_{\mathcal{D}}[\mathsf{guess}_{X\leftarrow\mathcal{D}}(X)] = \mathsf{guess}(X \mid \mathcal{D}) \leq \epsilon$ for some (small but fixed) $\epsilon > 0$; specifically, $\mathcal{D}$ can be described as a circuit that cannot make any queries to* $\mathsf{P3FQ}$, $\mathsf{P3FQ}^{-1}$*, or its internal round functions. Then, we have*

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}, \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}\right)\\ &\leq 88q_P\sqrt[3]{nq_{\mathcal{R}}\log|\mathcal{Z}|\,(\epsilon+2^{-n})} + 96nq_P(q_w+q_P-1)(\epsilon+2^{-n})\,. \qquad (16)\end{aligned}$$

Proof for $q_P = 1$. We first address the case where $q_P = 1$, i.e., $\mathcal{A}$ makes at most one query to O_b. Let $(s, \mathcal{D})$ be the input in that query, which is a random variable identically defined in both games (where $\mathcal{A}$ interacts with O_0 and O_1, respectively).Consider an intermediate hybrid between the games $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z)$ and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)$ where we replace the oracles O_0/O_1 with Hyb such that $\mathsf{Hyb}(1,\mathcal{D})$ outputs $O_0(1,\mathcal{D})$ and $\mathsf{Hyb}(-1,\mathcal{D})$ outputs $O_1(-1,\mathcal{D})$. It is then not hard to see that

$$\begin{aligned}&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\right)\\ &\leq \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z)\right)\\ &+ \mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},\mathsf{Hyb},\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\right)\\ &\leq 2\left(44\sqrt{nq_{\mathcal{R}}\log|\mathcal{Z}|\,(\epsilon+2^{-n})} + 48nq_w(\epsilon+2^{-n})\right),\end{aligned}$$

where we invoke Theorem 10 twice with respect to the jump from O_0 to Hyb, and Hyb to O_1. This concludes the proof for the $q_P = 1$ case. □

Proof for the general case. Extending the theorem from the $q_P = 1$ case to the general case follows a rather straightforward hybrid argument. Let Hyb_i for each $i \in \{0, \ldots, q_P\}$ be the game that is adapted from $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z)$, where we replace the oracle O_0 for the first i queries with the oracle O_1. Note that in the extreme case where $i = 0$ and where $i = q_P$, the games behave identically to $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z)$ and $\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)$ respectively. Hence, it suffices to show the closeness of $\mathsf{Hyb}_{i-1} \approx \mathsf{Hyb}_i$ for each $i \in [q_P]$, where the only difference is that the ith query to O_b, which we call the *crucial query*, is different — one is to O_0 while the other is to O_1. Indeed, before the crucial query, both Hyb_{i-1} and Hyb_i make at most q_w queries to Prog and at most $q_P - 1$ queries to O_1, each of which can be then be simulated using Prog. Therefore, applying the bound that we have obtained previously for $q_P = 1$ (substituting q_w with $q_w + q_P - 1$), we already obtain $\mathsf{SD}(\mathsf{Hyb}_{i-1}, \mathsf{Hyb}_i) \leq$

$88\sqrt{nq_{\mathcal{R}} \log |\mathcal{Z}| (\epsilon + 2^{-n})} + 96n(q_w + q_P - 1)(\epsilon + 2^{-n})$, which gives us

$$\begin{aligned}
&\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_0,\mathsf{Prog}}(z), \mathcal{A}^{\mathsf{P3FQ},\mathsf{P3FQ}^{-1},O_1,\mathsf{Prog}}(z)\right) \\
&\leq \sum_{i \in [q_P]} \mathsf{SD}(\mathsf{Hyb}_{i-1}, \mathsf{Hyb}_i) \\
&\leq q_P \left(88\sqrt{nq_{\mathcal{R}} \log |\mathcal{Z}| (\epsilon + 2^{-n})} + 96n(q_w + q_P - 1)(\epsilon + 2^{-n})\right) .
\end{aligned}$$

This concludes the proof for the general case. □

G Measure-Reprogram for Permutations

We show that our P3FQ Feistel tool is compatible with the *measure-and-reprogram technique*, first introduced in the QROM in [DFM20]. In the random oracle setting, the technique states that for any adversary $\mathcal{A}^H$ making q queries to a random oracle H and outputting a pair (x, z) such that some predicate $V(x, H(x), z)$ holds, there exists a simulator $\mathcal{S}$ with the following behavior: $\mathcal{S}$ first extracts x by *measuring* one query of $\mathcal{A}^H$ to H, then *reprograms* $H(x) \coloneqq \theta$ for a uniformly random θ. The output z of $\mathcal{A}^H$ then satisfies $V(x, \theta, z)$, up to a multiplicative $O(q^2)$ loss in probability.

Algorithm $\mathcal{A}$	Algorithm $\mathcal{S}$
1: $(x, z) \leftarrow \mathcal{A}^f$	1: $\theta \leftarrow\!\!\$\, \bar{\mathcal{Y}}$
	2: $(x_{\mathcal{S}}, st) \leftarrow \mathcal{S}^{\mathcal{A}}$
	3: $z_{\mathcal{S}} \leftarrow \mathcal{S}^{\mathcal{A}}(\theta, st)$

Fig. 9. Left: oracle (quantum) algorithm $\mathcal{A}$. We denote the acceptance probability of predicate V evaluated on $\mathcal{A}$'s output and the original function value $f(x)$ as $\Pr[V(x, f(x), z)]$. Right: algorithm $\mathcal{S}$ runs $\mathcal{A}$, simulating oracle $f : \bar{\mathcal{X}} \to \bar{\mathcal{Y}}$ towards it, and reprogramming $f(x)$ to θ. We denote the acceptance probability of predicate V evaluated on $\mathcal{S}$'s output and the reprogrammed value θ as $\Pr[V(x_{\mathcal{S}}, \theta, z_{\mathcal{S}})]$.

With the algorithms in Fig. 9 in mind we restate the technique for random oracles by [DFM20]:

Theorem 17 (Measure-and-reprogram (adapted [DFM20, Thm 2])). *Let $\mathcal{X}$ and $\mathcal{Y}$ be sets of all bit strings of a certain fixed length. Let $\mathcal{A}$ be an arbitrary oracle quantum algorithm that makes q_H queries to a random oracle $H : \mathcal{X} \to \mathcal{Y}$. Let $\mathcal{S}$ be a two-stage quantum algorithm that interacts with $\mathcal{A}$, and let V be an arbitrary predicate. The algorithms are depicted in Fig. 9, where we have $\bar{\mathcal{X}} = \mathcal{X}, \bar{\mathcal{Y}} = \mathcal{Y}$, $f = H$, and θ a fresh random value. We can then lower-bound the probability that the simulator $\mathcal{S}$ produces an output $(x_{\mathcal{S}}, z_{\mathcal{S}})$ such that the predicate V is satisfied with respect to the target value θ to the probability that V accepts on the adversary $\mathcal{A}$'s output (x, z) and associated $H(x)$. I.e., we have:*

$$\Pr[V(x_{\mathcal{S}}, \theta, z_{\mathcal{S}})] \geq \left(\Pr[V(x, H(x), z)]\right) / (2q_H + 1)^2 \ .$$

We now prove an analogous measure-and-reprogram technique for quantum-accessible random permutations using the P3FQ construction. In the following, we denote by $f[x \mapsto \theta]$ the oracle that is guaranteed to coincide with f everywhere except point x, which it maps to θ.

Theorem 18 (Measure-and-reprogram for P3FQ). *Let $\mathcal{X}$ be the set of all bit strings of a certain fixed length, and* P3FQ *be the permutation construction over $\mathcal{X}^2$ as defined in Fig. 1. Let $\mathcal{A}$ be an oracle quantum algorithm, and $\mathcal{S}$ be a two-stage quantum algorithm interacting with $\mathcal{A}$ defined w.r.t. some predicate V as shown in Fig. 9 where $\bar{\mathcal{X}} = \bar{\mathcal{Y}} = \mathcal{X}^2$ and $f = \{\mathsf{P3FQ}, \mathsf{P3FQ}^{-1}\}$. Let q be the number of queries to* P3FQ *and its inverse in total.*

We can then relate the probability that the simulation $\mathcal{S}$ outputs $(x_{\mathcal{S}}, z_{\mathcal{S}})$ s.t. the predicate V holds w.r.t. the target value θ with the probability that V accepts on the adversary $\mathcal{A}$'s output (x, z) and associated P3FQ(x) *with:*

$$\Pr\left[V(x_{\mathcal{S}}, \theta, z_{\mathcal{S}})\right] \geq \left(\Pr\left[V(x, \mathsf{P3FQ}(x), z)\right] - \sqrt{\frac{2q}{|\mathcal{X}|}} - \frac{q}{|\mathcal{X}|}\right)(4q+1)^4 \ .$$

Proof. We now present an explicit simulation procedure for $\mathcal{S}$ that simulates the P3FQ oracle for an adversary $\mathcal{A}$ defined with respect to a predicate V. The simulator $\mathcal{S}$ is given as input a reprogramming target $\theta = y' \in \mathcal{X}^2$.

Hyb_0**.** The simulator initializes the P3FQ construction using random permutations $\mathcal{P}, \mathcal{Q}$ (and their inverses $\mathcal{P}^{-1}, \mathcal{Q}^{-1}$), as well as functions H_1, H_2, H_3 uniformly at random on the set of functions on $\mathcal{X} \to \mathcal{X}$. This change is semantic.
Reprogramming strategy for H_1 and H_2. Secondly, the simulator replaces oracles H_i for $i \in \{1, 2\}$ with their reprogrammed variants, applying the measure-and-reprogram technique for QROs (Theorem 17) in hybrids Hyb_1 and Hyb_2. Intuitively, reprogramming each oracle H_i to θ_i works as follows:

1. the simulation samples an index $j \in [q_H - 1]$ and a bit $b \in \{0, 1\}$,
2. all queries up to the j-th query are answered using H_i,
3. at the j-th query, the simulation measures the adversary's query register to obtain the input x_i (for H_1, this is the L-wire), and
4. after the $(j + b)$-th query, uses the reprogrammed oracle $H_i[x_i \mapsto \theta_i]$.

We now describe the reprogramming for each oracle. Recall the setup: by definition, the adversary interacting with $\mathcal{S}$ eventually outputs some $x \in \mathcal{X}^2$ and some z, with respect to a predicate V defined on $(x, H_1(x), z)$. We split x into the Feistel wires by computing $\mathcal{Q}(x) =: (L, R)$. Then $x_1 = L$ is the input to H_1 and $x_2 = R \oplus H_1(L)$ is the input to H_2. Thus, upon obtaining the measurement results x_1 and x_2, the simulation can compute the permutation input as $x \leftarrow \mathcal{Q}^{-1}(L, R) = \mathcal{Q}^{-1}(x_1, x_2 \oplus H_1(x_1))$. We now show that, by applying this technique twice, the simulation can reprogram the L-wire. As before, $q_H = 2q$ since each query to P3FQ makes two queries to each H_i.

Hyb_1**.** We first apply the technique to H_1. The simulation samples an index $j \in [2q]$ and a bit b. On the j-th query to H_1, it measures the input register to obtain $x_1 = L$. $\mathcal{S}$ then chooses a fresh random θ_1 and answers all further queries

using $H_1[x_1 \mapsto \theta_1]$. This is the first step toward computing the preimage x. Since the R-wire has not yet been measured, this reprogramming cannot set the Feistel output R' to the desired target, so θ_1 must be random. Full reprogramming is achieved via adaptive reprogramming in hybrid Hyb_3. The probability that the simulation outputs an accepting (x, z) is at most a factor $(4q+1)^{-2}$ smaller than the adversary's.

Hyb_2. We make the analogous change to H_2: the simulation first samples an index $k \in [2q]$ and a bit b. On the k-th query, it measures the input register to H_2, obtaining x_2. It then sets $R \leftarrow \theta_1 \oplus x_2$ as the second half of the Feistel preimage. From L and R it computes $x = \mathcal{Q}^{-1}(L, R)$, which it outputs at the end of the first stage. In the second stage, after receiving the reprogramming target y', the simulator sets the target value for H_2 to $\theta_2 = L \oplus L'$. After the $(k+b)$-th query, it replaces H_2 with $H_2[x_2 \mapsto \theta_2]$. This incurs an additional multiplicative loss of at most $(4q+1)^{-2}$.

Hyb_3. At this point, the simulation is able to reprogram the output of the L-wire to an arbitrary value and outputs the desired pre-image (L, R) after its first stage. All that remains is to reprogram the R-wire. Because of the measurements in the previous hybrids, the input register x_3 is already measured. To obtain the desired reprogramming on both output wires, we set the output of H_3 at x_3 to $\theta_3 := R' \oplus x_2$. Since the input L' and target θ_3 are fresh uniformly random classical values, we can apply the adaptive reprogramming technique (Lemma 14) and obtain

$$\mathsf{SD}\left(\mathcal{A}^{\mathsf{P3FQ}_2}, \mathcal{A}^{\mathsf{P3FQ}_3}\right) \leq \sqrt{2q\,\mathsf{guess}(L')} + 2q\,\mathsf{guess}(L')/2 \leq \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|}\ .$$

With this final reprogramming, the simulation now behaves as desired. To conclude, this gives us the total bound

$$\Pr[V(x_{\mathcal{S}}, \theta, z_{\mathcal{S}}) : (x_{\mathcal{S}}, z_{\mathcal{S}}) \leftarrow \langle \mathcal{S}^{\mathcal{A}}, \theta \rangle] \geq \left(\Pr[V(x, \mathsf{P3FQ}(x), z) : (x, z) \leftarrow \mathcal{A}^{\mathsf{P3FQ}}] - \sqrt{\frac{2q}{|\mathcal{X}|}} - \frac{q}{|\mathcal{X}|}\right)/(4q+1)^4\ . \quad \square$$

G.1 Post-Quantum Soundness Preservation of DSFS

As an application of our P3FQ measure-and-reprogram, we prove post-quantum adaptive soundness of NIZKs obtained via the duplex-sponge Fiat-Shamir (DSFS) transformation [CO25] (see Section 5 and Fig. 4 for details of the transform).

In the notation "$(\mathrm{x}, b) \leftarrow \Sigma\langle \mathcal{A}, \mathcal{V} \rangle$", x is the instance chosen by the adaptive adversary $\mathcal{A}$ at the start, and $b \in \{0, 1\}$ is the output of $\mathcal{V}$ after running Σ with $\mathcal{A}$ acting as prover with instance x. We then define the adaptive soundness error of proof-system Σ as

$$\mathsf{KSE}_{\Sigma}(\mathcal{A}) := \Pr[(\mathrm{x}, 1) \leftarrow \Sigma\langle \mathcal{A}, \mathcal{V} \rangle \wedge \mathrm{x} \text{ is invalid}^{14}] \leq \kappa_{\Sigma}\ .$$

[14] Namely, there is no witness w such that the pair (x, w) is valid.

We define the adaptive soundness error for a non-interactive proof system as

$$\Pr[1 \leftarrow \mathsf{Ver}(\mathrm{x}, \pi) \wedge \mathrm{x} \text{ is invalid} | (\mathrm{x}, \pi) \leftarrow \mathcal{A}] \leq \kappa_{\mathsf{DSFS}} \ .$$

For the sake of simplicity, we prove soundness preservation for the case of three round protocols ($k = 2$). The proof can be extended to multi-round protocols at the cost of additional reprogrammings.

Theorem 19 (Post-quantum (adaptive) soundness preservation of DSFS). *Let $\mathcal{X}^2$ be the set of all $2n$-bit strings, for $n \in N$. Let $\mathcal{R} : \mathcal{X}^2 \to \mathcal{X}^2$ be a random permutation. Let q be the number of possibly quantum queries to $\mathcal{R}$. Let Σ be a 3-round public-coin proof-system with soundness error κ_Σ. Then $\mathsf{DSFS}[\Sigma, \mathcal{R}]$ is a non-interactive proof system for adaptive adversaries with soundness error*

$$\kappa_{\mathsf{DSFS}} \leq (4q+1)^4 \left((4q+1)^4 \left(\kappa_\Sigma + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right) + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right) + 2^{-2n}$$

Proof. The high-level intuition is that we want to build a reduction $\mathcal{S}$ that uses an attacker $\mathcal{A}_{\mathsf{DSFS}}$ against the soundness of DSFS to break soundness of Σ. For this, the idea is that $\mathcal{S}$ extracts a_1 from $\mathcal{A}_{\mathsf{DSFS}}$'s query to $\mathcal{R}$ that they need to make to obtain the challenge ch_1. This is done using measure-and-reprogram. If the guess is correct, $\mathcal{S}$ sends the extracted a_1 as its own first message to Ver, receives a challenge ch and programs $\mathcal{R}$ to return ch for the measured query.

A minor complication is that the query to $\mathcal{R}$ that contains a_1 is actually $((a_1 \| 0^{2n-r}) \oplus s_1)$, i.e., a_1 is masked by s_1 which is unknown to $\mathcal{S}$ (c.f., Eq. (6)). Consequently, we need another round of measure-and-reprogram to obtain s_1[15]. Finally, the above requires us to replace $\mathcal{R}$ by P3FQ, hence we need to move to an efficient simulation of that in the end.

In more detail, we define $\mathcal{S}$ to be a two-stage quantum algorithm that simulates the soundness game for an adaptive DSFS malicious prover $\mathcal{A}_{\mathsf{DSFS}}$.

Step_1. In a first step, $\mathcal{S}$ replaces $\mathcal{R}$ with P3FQ. By Lemma 1, we have that the success probability of the adversary is unaffected.

Step_2. Next, the simulator measures one of the queries $\mathcal{A}_{\mathsf{DSFS}}$ makes to $\mathcal{R}$ (now P3FQ) to obtain s_1. This value is required so that, when another query is measured later, one can compute a_1. Since this step is solely aimed at obtaining a measurement result, the reprogramming target is chosen as a fresh random value θ_1. Hence, by applying Theorem 18, and using the fact that the condition requiring the adversary to output the preimages τ and x is satisfied, we obtain

$$\mathsf{KSE}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{S}^{\mathcal{A}}_{\mathsf{Hyb}_1}) \leq (4q+1)^4 \left(\mathsf{KSE}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{S}^{\mathcal{A}}_{\mathsf{Hyb}_2}) + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right).$$

Step_3. In the next step, $\mathcal{S}$ measures a random query $\mathcal{A}_{\mathsf{DSFS}}$ makes to $\mathcal{R}$ (now P3FQ), parsing it as a_1 later used in the forgery (line 6 in the $\mathsf{Prv}^{\mathcal{R}}$ algorithm in

[15] One may get the idea that this could also be done using the extractable QROM [DFMS22]. Sadly this is not the case, even if we did lift it to P3FQ, as we do not have a function value which we can use for extraction.

Fig. 4) using the previously obtained s_1 (since the input to $\mathcal{R}$ is $((a_1 \| 0^{2n-r}) \oplus s_1)$). It gets $\mathsf{ch}_1 \leftarrow \mathsf{Ver}(a_1)$ as the output of the verifier on input a_1 and combines it with a fresh random s_2 to use as reprogramming target $\theta_2 := (\mathsf{ch}_1; s_2)$. When $\mathcal{A}_{\mathsf{DSFS}}$ outputs proof $\pi = (\tau, a_1, \mathsf{ch}_1, a_2)$, $\mathcal{S}$ extracts a_2 and sends it to Ver. Since the preconditions for the measure-reprogram technique are met, we have

$$\mathsf{KSE}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{S}^{\mathcal{A}}_{\mathsf{Hyb}_2}) \leq (4q+1)^4 \left(\mathsf{KSE}_{\mathsf{DSFS}[\Sigma,\mathcal{R}]}(\mathcal{S}^{\mathcal{A}}_{\mathsf{Hyb}_3}) + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right).$$

$\mathsf{Step_4}$. In a last step, $\mathcal{S}$ switches to simulating $\mathsf{P3FQ}$ following Appendix A. This adds an additive 2^{-2n} error to the success probability and increases the runtime by $O(q_{\mathcal{R}}^2 \cdot n^4)$. Putting together the terms, we have that

$$\kappa'_{\mathsf{DSFS}} \leq (4q+1)^4 \left((4q+1)^4 \left(\kappa_\Sigma + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right) + \sqrt{\frac{2q}{|\mathcal{X}|}} + \frac{q}{|\mathcal{X}|} \right) + 2^{-2n} \ .$$

□

$\mathsf{Prv}^{\mathsf{P3FQ}}(\mathrm{x}, \mathrm{w})$:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $\tau \leftarrow \{0,1\}^r$
3: $s_1 := \mathsf{P3FQ}((\tau \| 0^{2n-r}) \oplus s_0)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
7: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
8: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
9: **return** $(\tau, a_1, \ldots, a_k)$

Hyb_0:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: $\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
7: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
8: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
9: **return** $(\tau, a_1, \ldots, a_k)$

Hyb_J:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: $\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad$ **if** $i < J$ **then**
7: $\quad\quad s_{i+1} \leftarrow \{0,1\}^{2n}$
8: $\quad\quad \mathsf{Prog}((a_i \| 0^{2n-r}) \oplus s_i, s_{i+1})$
9:
10:
11: $\quad$ **else** $s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
12: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
13: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
14: **return** $(\tau, a_1, \ldots, a_k)$

rewritten Hyb_J:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: $\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad$ **if** $i < J$ **then**
7: $\quad\quad s_{i+1} \leftarrow \{0,1\}^{2n}$
8: $\quad\quad \mathsf{Prog}((a_i \| 0^{2n-r}) \oplus s_i, s_{i+1})$
9: $\quad$ **else if** $i = J$ **then**
10: $\quad\quad s_{J+1} := \mathsf{P3FQ}((a_J \| 0^{2n-r}) \oplus s_J)$
11: $\quad$ **else** $s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
12: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
13: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
14: **return** $(\tau, a_1, \ldots, a_k)$

Hyb'_J:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: ~~$\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$~~
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad$ **if** $i < J$ **then**
7: $\quad\quad s_{i+1} \leftarrow \{0,1\}^{2n}$
8: $\quad\quad$ ~~$\mathsf{Prog}((a_i \| 0^{2n-r}) \oplus s_i, s_{i+1})$~~
9: $\quad$ **else if** $i = J$ **then**
10: $\quad\quad s_{J+1} := \mathsf{P3FQ}((a_J \| 0^{2n-r}) \oplus s_J)$
11: $\quad\quad \mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
12: $\quad\quad$ **for** $j = 1, \ldots J-1$
13: $\quad\quad\quad \mathsf{Prog}((a_j \| 0^{2n-r}) \oplus s_j, s_{j+1})$
14: $\quad$ **else** $s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
15: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
16: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
17: **return** $(\tau, a_1, \ldots, a_k)$

Hyb''_J:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \leftarrow \{0,1\}^{2n}$
3:
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad$ **if** $i < J$ **then**
7: $\quad\quad s_{i+1} \leftarrow \{0,1\}^{2n}$
8:
9: $\quad$ **else if** $i = J$ **then**
10: $\quad\quad \mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1} \leftarrow \{0,1\}^{2n})$
11: $\quad\quad \mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
12: $\quad\quad$ **for** $j = 1, \ldots J-1$
13: $\quad\quad\quad \mathsf{Prog}((a_j \| 0^{2n-r}) \oplus s_j, s_{j+1})$
14: $\quad$ **else** $s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
15: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
16: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
17: **return** $(\tau, a_1, \ldots, a_k)$

Hyb'''_J:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: $\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad$ **if** $i < J$ **then**
7: $\quad\quad s_{i+1} \leftarrow \{0,1\}^{2n}$
8: $\quad$ **else if** $i = J$ **then**
9: $\quad\quad$ ~~$\mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1} \leftarrow \{0,1\}^{2n})$~~
10: $\quad\quad \mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
11: $\quad\quad$ **for** $j = 1, \ldots J-1$
12: $\quad\quad\quad \mathsf{Prog}((a_j \| 0^{2n-r}) \oplus s_j, s_{j+1})$
13: $\quad\quad \mathsf{Prog}((a_J \| 0^{2n-r}) \oplus s_J, s_{J+1} \leftarrow \{0,1\}^{2n})$
14: $\quad$ **else** $s_{i+1} := \mathsf{P3FQ}((a_i \| 0^{2n-r}) \oplus s_i)$
15: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
16: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
17: **return** $(\tau, a_1, \ldots, a_k)$

$\mathsf{Trans}^{\mathsf{P3FQ},\mathsf{Prog}}(\mathrm{x}, \mathrm{w})$:
1: $s_0 := \mathsf{P3FQ}(\mathrm{x} \| 0^{2n-r})$
2: $(\tau, s_1) \leftarrow \{0,1\}^r \times \{0,1\}^{2n}$
3: $\mathsf{Prog}((\tau \| 0^{2n-r}) \oplus s_0, s_1)$
4: $(a_1, \mathsf{st}_1) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w})$
5: **for** $i = 1, \ldots, k-1$
6: $\quad \mathsf{Prog}((a_i \| 0^{2n-r}) \oplus s_i, s_{i+1} \leftarrow \{0,1\}^{2n})$
7: $\quad \mathsf{ch}_i := s_{i+1}|_{1,\ldots,r}$
8: $\quad (a_{i+1}, \mathsf{st}_{i+1}) \leftarrow P_\Sigma(\mathrm{x}, \mathrm{w}, \mathsf{st}_i, \mathsf{ch}_i)$
9: **return** $(\tau, a_1, \ldots, a_k)$

Fig. 6. Oracles used in our hybrid sequence for proving Theorem 11. Here $J \in [k]$.